

Fundamentos de programación con Python

[GUÍA RÁPIDA DE ESTUDIO]

@garydevelop

GM

Índice

Parte 1 · Bases de la programación	4	Parte 4 · Listas y funciones generales	25
1 Programas	4	38 Tipo de dato: lista	25
2 Editores y compiladores	4	39 Slicing de listas	25
3 Algoritmos	5	40 Operadores en listas	26
4 Lenguajes de programación	5	41 Métodos de listas	27
5 Python	6	42 Métodos split() y join()	28
6 Intérpretes	6	43 Desempaquetado	28
7 Programación	7	44 Funciones generales	29
Parte 2 · Datos, variables y operadores	8	45 Módulo random	30
8 Tipos de datos	8	46 Listas anidadas (matrices)	30
9 Variables	8	47 Comprensión de listas	31
10 Nombres de variables	9	Parte 5 · Control de flujo y funciones	32
11 Buenas prácticas	9	48 Sentencias condicionales	32
12 Palabras reservadas	10	49 Bucle for	33
13 Operadores aritméticos	10	50 Bucle while	34
14 Operadores relacionales	11	51 Creación de funciones	35
15 Operadores lógicos	11	52 Excepciones (try / except)	36
16 Operadores de incremento y decremento	12	Parte 6 · Archivos y estructuras de datos	38
17 Módulo math	13	53 Archivos	38
18 Entrada y salida de datos	13	54 Tipo de dato: tupla	39
19 Conversiones	14	55 Tipo de dato: diccionario	40
20 Errores comunes	15	56 Funciones de diccionarios	41
21 Format (f-strings)	15	57 Conjuntos (set)	42
22 Decimales y redondeo	16	Parte 7 · Ciencia de datos: NumPy, Pandas y Matplotlib	43
Parte 3 · Cadenas de texto (str)	17	58 NumPy: arreglos	43
23 Tipo de dato: string	17	59 NumPy: operaciones numéricas	44
24 Index de str	17	60 NumPy: estadísticas y cálculos eficientes	45
25 Slicing de str	18	61 NumPy: propiedades, funciones y métodos en vectores	45
26 Operaciones de comparación	18	62 NumPy: indexación de posiciones	46
27 Secuencias de escape	19	63 NumPy: comparación e indexación booleana	47
28 Tabulador	19	64 Pandas: Series y DataFrame	48
29 Operadores en str	20	65 Pandas: cargar datos (CSV y Excel)	49
30 Operador in	20	66 Pandas: explorar y limpiar	50
31 Funciones de str (formato)	21	67 Pandas: seleccionar y filtrar	51
32 Longitud de cadena	21	68 Pandas: agrupar y analizar	53
33 Contar con count()	22	69 Matplotlib: gráficos básicos	54
34 Reemplazar con replace()	22	70 Matplotlib: personalizar y guardar	55
35 Cambiar mayúsculas y minúsculas	23		
36 Buscar en un texto	23		
37 Funciones que comienzan con is...	24		

Cómo usar esta guía

Una guía rápida de **Fundamentos de Programación con Python**: conceptos cortos, trucos para recordarlos y tablas con ejemplos listos para copiar.

1 · Lee el concepto

Dos líneas que explican de qué trata el tema, sin vueltas.

2 · Memoriza el truco

La caja **RECUERDA** trae una comparación fácil para no olvidar la idea.

3 · Prueba la tabla

Cada fila trae **función, descripción, ejemplo y resultado**. Cópialo y córrelo tú mismo.

Cómo leer una tabla

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>len()</code>	Qué hace la función, en una frase.	<code>len("hola")</code>	4

Mapa de la guía

PARTE 1 Bases de la programación

pág. 4

Qué es un programa, cómo se escribe y cómo lo entiende la computadora. (7 temas)

PARTE 2 Datos, variables y operadores

pág. 8

Los ladrillos básicos: tipos, variables, operadores y cómo leer y mostrar datos. (15 temas)

PARTE 3 Cadenas de texto (str)

pág. 17

Cómo guardar, recorrer, comparar y transformar texto. (15 temas)

PARTE 4 Listas y funciones generales

pág. 25

Guardar varios datos juntos y las herramientas para trabajar con ellos. (10 temas)

PARTE 5 Control de flujo y funciones

pág. 32

Tomar decisiones, repetir tareas y organizar el código en funciones. (5 temas)

PARTE 6 Archivos y estructuras de datos

pág. 38

Guardar información en el disco y organizar datos con tuplas, diccionarios y conjuntos. (5 temas)

PARTE 7 Ciencia de datos: NumPy, Pandas y Matplotlib

pág. 43

Las tres librerías base para analizar números, tablas y gráficos. (13 temas)

Antes de empezar

- Todos los resultados de las tablas salieron de **ejecutar el código de verdad** (Python 3.11, NumPy 2.4, Pandas 3.0, Matplotlib 3.10). Si usas otra versión, algún detalle de formato puede verse distinto.
- Las filas con «Ej:» usan números al azar: te saldrá otro valor cada vez.
- Cuando ves **datos de ejemplo** arriba de una tabla (por ejemplo `nums = [4, 1, 3]`), esas variables ya existen en todos los ejemplos del tema.
- Los errores se muestran como en Python: nombre del error y mensaje. En tu consola verás además la línea donde ocurrió.

Material de estudio no oficial, hecho por estudiantes para estudiar. No reemplaza las clases ni el material oficial de la materia.

Bases de la programación

Qué es un programa, cómo se escribe y cómo lo entiende la computadora.

TEMA 1

Programas

Un programa es una lista de instrucciones que la computadora ejecuta en orden, de arriba hacia abajo. Recibe datos de entrada, los procesa y entrega una salida.

RECUERDA Entrada → Proceso → Salida. Como una licuadora: metes las frutas, la enciendes y sale el jugo.

INSTRUCCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>print()</code>	Muestra un mensaje en pantalla (salida).	<code>print("Hola, GM")</code>	Hola, GM
<code>input()</code>	Pide un dato al usuario (entrada). Siempre devuelve texto.	<code>nombre = input("Tu nombre: ")</code> <code>print("Hola", nombre)</code>	Tu nombre: Ana Hola Ana
# comentario	Nota para humanos. Python la ignora.	<code># esto no se ejecuta</code> <code>print("listo")</code>	listo
Programa completo	Entrada, proceso y salida juntos.	<code>a = int(input("A: "))</code> <code>b = int(input("B: "))</code> <code>print("Suma:", a + b)</code>	A: 4 B: 6 Suma: 10
Orden de ejecución	Python lee línea por línea, de arriba hacia abajo.	<code>print(1)</code> <code>print(2)</code> <code>print(3)</code>	1 2 3

TEMA 2

Editores y compiladores

Un editor es donde escribes el código (VS Code, Spyder, Jupyter, IDLE). Un compilador traduce todo el programa a lenguaje máquina antes de ejecutarlo (C, Java). Python usa un intérprete que ejecuta línea por línea.

RECUERDA Editor = cuaderno donde escribes. Compilador = traductor del libro completo. Intérprete = traductor en vivo, frase por frase.

HERRAMIENTA	DESCRIPCIÓN	EJEMPLO	RESULTADO
VS Code	Editor gratuito y ligero, con extensiones para Python.	<code>code hola.py</code>	Abre hola.py en VS Code
Jupyter Notebook	Ejecuta el código por celdas. Ideal para datos.	<code>jupyter notebook</code>	Abre el cuaderno en el navegador
Ejecutar un script	El intérprete lee y ejecuta el archivo .py (si hola.py tiene <code>print("Hola, GM")</code>).	<code>python hola.py</code>	Hola, GM
Compilar en C	El compilador genera un ejecutable antes de correr.	<code>gcc hola.c -o hola</code>	Crea el archivo ejecutable hola
IDE	Editor + ejecutar + depurar en un solo programa (PyCharm, Spyder).	Run ►	Ejecuta el programa y muestra la salida abajo

TEMA 3

Algoritmos

Un algoritmo es una serie de pasos ordenados y finitos para resolver un problema. Primero se piensa el algoritmo y después se escribe en un lenguaje de programación.

RECUERDA Algoritmo = receta de cocina: pasos claros, en orden y que terminan.

PARTE	DESCRIPCIÓN	EJEMPLO	RESULTADO
1. Entrada	Los datos con los que empieza.	<code>base = 6 altura = 4 print(base, altura)</code>	6 4
2. Proceso	Cálculos o decisiones con esos datos.	<code>base = 6 altura = 4 area = base * altura / 2 print(area)</code>	12.0
3. Salida	El resultado que se muestra.	<code>area = 12.0 print("Área:", area)</code>	Área: 12.0
Decisión	Elegir un camino según una condición.	<code>n = 7 print("par" if n % 2 == 0 else "impar")</code>	impar
Repetición	Hacer algo varias veces.	<code>for i in range(3): print("paso", i)</code>	paso 0 paso 1 paso 2
Pseudocódigo	Escribir los pasos en español antes de programar.	Leer n Si n es par: mostrar "par"	Plan listo para pasarlo a Python

TEMA 4

Lenguajes de programación

Es el idioma con el que le damos instrucciones a la computadora. Los de bajo nivel están cerca de la máquina; los de alto nivel están cerca del lenguaje humano, como Python.

RECUERDA Bajo nivel = hablar el idioma de la máquina. Alto nivel = que la máquina te entienda a ti.

LENGUAJE	DESCRIPCIÓN	EJEMPLO	RESULTADO
Python	Alto nivel, interpretado y muy legible.	<code>print("Hola")</code>	Hola
C	Compilado y rápido. Cercano al hardware.	<code>printf("Hola");</code>	Hola
Java	Se compila a bytecode. Muy usado en empresas.	<code>System.out.println("Hola");</code>	Hola
JavaScript	El lenguaje de las páginas web.	<code>console.log("Hola");</code>	Hola
SQL	Para consultar bases de datos.	<code>SELECT nombre FROM alumnos;</code>	Lista los nombres de los alumnos

TEMA 5

Python

Lenguaje de alto nivel, interpretado y de sintaxis simple. Se usa en ciencia de datos, inteligencia artificial, web y automatización. La indentación (espacios al inicio de la línea) es parte de la sintaxis.

RECUERDA Python = lenguaje «casi como el inglés»: si lo lees en voz alta, se entiende.

CARACTERÍSTICA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Sintaxis simple	Sin llaves ni punto y coma al final.	<code>print("Hola")</code>	Hola
Indentación	Los espacios definen qué línea está dentro del bloque.	<code>if True: print("dentro") print("fuera")</code>	dentro fuera
Tipado dinámico	No declaras el tipo, Python lo deduce.	<code>x = 5 x = "cinco" print(type(x))</code>	<class 'str'>
Versión	Muestra la versión principal instalada.	<code>import sys print(sys.version_info.major)</code>	3
Modo calculadora	Python evalúa operaciones directamente.	<code>2 ** 10</code>	1024
Sensible a mayúsculas	print no es lo mismo que Print.	<code>Print("hola")</code>	NameError: name 'Print' is not defined

TEMA 6

Interpretes

El intérprete lee tu código y lo ejecuta línea por línea. Si hay un error, se detiene justo en esa línea. CPython es el intérprete oficial de Python.

RECUERDA Intérprete = traductor en vivo: traduce una frase, la ejecuta y pasa a la siguiente.

MODO / HERRAMIENTA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Modo interactivo	Consola (>>>) que evalúa cada línea al instante.	<code>3 + 4</code>	7
Script .py	Ejecuta el archivo completo.	<code>python programa.py</code>	Corre todo el archivo
Error en una línea	Se detiene donde falla. Lo anterior sí se ejecutó.	<code>print("antes") print(1 / 0) print("después")</code>	antes ZeroDivisionError: division by zero
Qué intérprete uso	Nombre de la implementación.	<code>import platform print(platform.python_implementation())</code>	CPython
Salir de la consola	Cierra el modo interactivo.	<code>exit()</code>	Cierra la consola

Programación

Programar es convertir un algoritmo en instrucciones que la computadora entiende. La sintaxis son las reglas de escritura; la semántica es lo que realmente hace el código.

RECUERDA Sintaxis = gramática (¿está bien escrito?). Semántica = significado (¿hace lo que quiero?).

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Instrucción	Una orden para la computadora.	<code>print("Hola")</code>	Hola
Error de sintaxis	El código está mal escrito.	<code>print("Hola"</code>	SyntaxError: '(' was never closed
Error de semántica	Corre sin error, pero ¿querías $(2 + 3) * 4 = 20$? Python multiplica primero.	<code>print(2 + 3 * 4)</code>	14
Bloque	Instrucciones agrupadas por indentación.	<code>for i in range(2): print(i)</code>	0 1
Línea larga	Una instrucción en varias líneas con paréntesis.	<code>total = (1 + 2 + 3) print(total)</code>	6
Comentario	Explica el código. No se ejecuta.	<code># calcula el doble print(4 * 2)</code>	8

Datos, variables y operadores

Los ladrillos básicos: tipos, variables, operadores y cómo leer y mostrar datos.

TEMA 8

Tipos de datos

El tipo de dato dice qué clase de información guarda un valor y qué se puede hacer con ella. Los básicos son int (enteros), float (decimales), str (texto) y bool (verdadero o falso).

RECUERDA int = contar manzanas. float = pesar manzanas (decimales). str = escribir la palabra «manzana». bool = ¿hay manzanas? Sí o No.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
int	Número entero.	<code>type(7)</code>	<code><class 'int'></code>
float	Número con decimales.	<code>type(3.14)</code>	<code><class 'float'></code>
str	Texto entre comillas.	<code>type("hola")</code>	<code><class 'str'></code>
bool	Solo True o False.	<code>type(True)</code>	<code><class 'bool'></code>
list	Colección ordenada y modificable.	<code>type([1, 2])</code>	<code><class 'list'></code>
tuple	Colección ordenada e inmutable.	<code>type((1, 2))</code>	<code><class 'tuple'></code>
dict	Pares clave: valor.	<code>type({"a": 1})</code>	<code><class 'dict'></code>
None	Valor vacío, «sin valor».	<code>type(None)</code>	<code><class 'NoneType'></code>
type()	Dice el tipo de un valor. Ojo: 5 / 2 da float.	<code>type(5 / 2)</code>	<code><class 'float'></code>
isinstance()	Pregunta si un valor es de cierto tipo.	<code>isinstance(5, int)</code>	True

TEMA 9

Variables

Una variable es una caja con nombre donde guardas un valor. Se crea con = (asignación) y puedes cambiar su contenido cuando quieras.

RECUERDA Variable = caja con etiqueta. La etiqueta es el nombre y lo que hay adentro es el valor.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Asignación =	Guarda un valor en una variable.	<code>edad = 19</code> <code>print(edad)</code>	19
Reasignar	Cambia el valor guardado.	<code>x = 5</code> <code>x = 8</code> <code>print(x)</code>	8
Copiar valor	Copiar no enlaza: cambiar una no cambia la otra.	<code>a = 10</code> <code>b = a</code> <code>a = 99</code> <code>print(b)</code>	10
Asignación múltiple	Varias variables en una línea.	<code>x, y = 3, 4</code> <code>print(x, y)</code>	3 4
Mismo valor	Mismo valor para varias variables.	<code>a = b = 0</code> <code>print(a, b)</code>	0 0

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Intercambiar	Cambia dos valores de lugar.	<code>a, b = 1, 2</code> <code>a, b = b, a</code> <code>print(a, b)</code>	2 1
del	Borra una variable.	<code>x = 5</code> <code>del x</code> <code>print(x)</code>	NameError: name 'x' is not defined
Variable sin crear	Usarla antes de asignarla da error.	<code>print(precio)</code>	NameError: name 'precio' is not defined

TEMA 10

Nombres de variables

Reglas: empiezan con letra o guion bajo, solo llevan letras, números y `_`, no tienen espacios, no pueden ser palabras reservadas y Python distingue mayúsculas de minúsculas.

RECUERDA El nombre es como un DNI claro: sin espacios, sin empezar con número, y «Edad» no es lo mismo que «edad».

NOMBRE	DESCRIPCIÓN	EJEMPLO	RESULTADO
edad	Válido: minúsculas.	<code>edad = 18</code> <code>print(edad)</code>	18
_total	Válido: puede empezar con guion bajo.	<code>_total = 5</code> <code>print(_total)</code>	5
nota_1	Válido: números al final.	<code>nota_1 = 9</code> <code>print(nota_1)</code>	9
1nota	Inválido: empieza con número.	<code>1nota = 9</code>	SyntaxError: invalid decimal literal
mi nota	Inválido: tiene espacio.	<code>mi nota = 9</code>	SyntaxError: invalid syntax
mi-nota	Inválido: el guion es una resta.	<code>mi-nota = 9</code>	SyntaxError: cannot assign to expression here. Maybe you meant '==' instead of '='?
class	Inválido: palabra reservada.	<code>class = 5</code>	SyntaxError: invalid syntax
Edad ≠ edad	Las mayúsculas importan.	<code>Edad = 18</code> <code>print(edad)</code>	NameError: name 'edad' is not defined

TEMA 11

Buenas prácticas

Código limpio es el que otros (y tú dentro de una semana) pueden entender: nombres claros, comentarios útiles, espacios ordenados y 4 espacios de indentación. Se sigue la guía PEP 8.

RECUERDA Escribe código para humanos, no solo para la máquina. Si no se entiende solo, cámbiale el nombre.

PRÁCTICA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Nombres claros	El nombre explica el dato.	<code>precio_final = round(80 * 1.12, 2)</code> <code>print(precio_final)</code>	89.6
snake_case	Minúsculas con <code>_</code> entre palabras.	<code>nota_final = 8.5</code> <code>print(nota_final)</code>	8.5
Constantes	Valores fijos en MAYÚSCULAS.	<code>IVA = 0.12</code> <code>print(100 * IVA)</code>	12.0
Espacios	Un espacio alrededor de los operadores: <code>x = 3 + 4</code> .	<code>x = 3 + 4</code> <code>print(x)</code>	7

PRÁCTICA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Comentarios útiles	Explican el porqué, no lo obvio.	<pre># pasa de °C a °F c = 100 f = c * 9 / 5 + 32 print(f)</pre>	212.0
Sin números mágicos	Guarda el número en una variable con nombre.	<pre>DIAS_SEMANA = 7 print(4 * DIAS_SEMANA)</pre>	28
Indentación	4 espacios por nivel. Nunca mezcles con tabs.	<pre>if True: print("ok")</pre>	ok

TEMA 12

Palabras reservadas

Son palabras que Python usa para su propia sintaxis y no puedes usarlas como nombres de variables. Se consultan con el módulo keyword.

RECUERDA Si el editor las pinta de otro color, son reservadas: no las uses como nombre.

PALABRA	DESCRIPCIÓN	EJEMPLO	RESULTADO
keyword.kwlist	Lista de palabras reservadas (primeras 5).	<pre>import keyword print(keyword.kwlist[:5])</pre>	['False', 'None', 'True', 'and', 'as']
keyword.iskeyword()	Pregunta si una palabra es reservada.	<pre>import keyword print(keyword.iskeyword("for"))</pre>	True
if, elif, else	Decisiones.	<pre>if 3 > 2: print("sí")</pre>	sí
for, while	Repeticiones.	<pre>for i in range(2): print(i)</pre>	0 1
def, return	Crear funciones.	<pre>def doble(n): return n * 2 print(doble(4))</pre>	8
True, False, None	Valores especiales.	<pre>print(True, False, None)</pre>	True False None
and, or, not	Operadores lógicos.	<pre>print(True and not False)</pre>	True
in, is	Pertenencia e identidad.	<pre>print(3 in [1, 2, 3])</pre>	True
import	Traer módulos.	<pre>import math print(math.sqrt(9))</pre>	3.0
print no es reservada	Es una función. No la uses de nombre.	<pre>import keyword print(keyword.iskeyword("print"))</pre>	False

TEMA 13

Operadores aritméticos

Hacen cálculos: + suma, - resta, * multiplicación, / división (siempre da float), // división entera, % residuo y ** potencia. Orden: paréntesis, luego **, luego * / // %, y al final + -.

RECUERDA % es el «sobrante»: repartes 7 caramelos entre 3 niños, tocan 2 a cada uno (//) y sobra 1 (%).

OPERADOR	DESCRIPCIÓN	EJEMPLO	RESULTADO
+	Suma.	8 + 5	13
-	Resta.	8 - 5	3
*	Multiplicación.	8 * 5	40
/	División. Siempre devuelve float.	7 / 2	3.5

OPERADOR	DESCRIPCIÓN	EJEMPLO	RESULTADO
//	División entera (descarta decimales).	17 // 5	3
%	Residuo de la división.	17 % 5	2
**	Potencia.	2 ** 5	32
Raíz con **	Elevar a 0.5 es raíz cuadrada.	16 ** 0.5	4.0
Jerarquía	Primero la multiplicación.	2 + 3 * 4	14
Paréntesis	Cambian el orden.	(2 + 3) * 4	20
Par o impar	n % 2 da 0 si es par.	10 % 2	0
divmod()	Cociente y residuo juntos.	divmod(17, 5)	(3, 2)

TEMA 14

Operadores relacionales

Comparan dos valores y devuelven True o False: == igual, != distinto, > mayor, < menor, >= mayor o igual, <= menor o igual.

RECUERDA Un solo = guarda un valor. Dos == hacen una pregunta.

OPERADOR	DESCRIPCIÓN	EJEMPLO	RESULTADO
==	¿Son iguales?	5 == 5	True
!=	¿Son distintos?	5 != 3	True
>	¿Mayor que?	7 > 10	False
<	¿Menor que?	7 < 10	True
>=	¿Mayor o igual?	7 >= 7	True
<=	¿Menor o igual?	8 <= 7	False
Encadenados	Varias comparaciones juntas.	1 < 5 < 10	True
Textos	Se comparan letra por letra.	"a" < "b"	True
Ojo con decimales	Los float tienen error mínimo.	0.1 + 0.2 == 0.3	False
Guardar el resultado	El resultado es un bool.	aprobado = 8 >= 7 print(aprobado)	True

TEMA 15

Operadores lógicos

Combinan condiciones: and (las dos deben ser verdaderas), or (basta con una) y not (invierte el valor).

RECUERDA and = «y» estricto: los dos. or = «o» flexible: con uno basta. not = lo contrario.

OPERADOR	DESCRIPCIÓN	EJEMPLO	RESULTADO
and	True solo si ambas son True.	True and False	False
or	True si al menos una es True.	True or False	True
not	Invierte el valor.	not True	False
En una condición	Edad entre 18 y 64.	edad = 20 print(edad >= 18 and edad < 65)	True
not con comparación	Niega el resultado.	not (3 > 5)	True

OPERADOR	DESCRIPCIÓN	EJEMPLO	RESULTADO
Cortocircuito	Si el primero ya decide, no evalúa el resto.	<code>False and 1 / 0</code>	False
Valores «falsos»	0, "", [] y None cuentan como False.	<code>print(bool(0), bool(""), bool([]))</code>	False False False
Valores «verdaderos»	Casi todo lo demás cuenta como True.	<code>print(bool(5), bool("hola"), bool([0]))</code>	True True True
or como valor por defecto	Devuelve el primero que sea verdadero.	<code>"" or "sin nombre"</code>	'sin nombre'

TEMA 16

Operadores de incremento y decremento

Python no tiene ++ ni --. Para aumentar o disminuir una variable se usan += y -= (también *=, /=, //=, %= y **=).

RECUERDA x += 1 se lee «x es x más uno». Perfecto para contadores y acumuladores.

OPERADOR	DESCRIPCIÓN	EJEMPLO	RESULTADO
+=	Suma y guarda.	<code>x = 10 x += 5 print(x)</code>	15
-=	Resta y guarda.	<code>x = 10 x -= 3 print(x)</code>	7
*=	Multiplica y guarda.	<code>x = 10 x *= 2 print(x)</code>	20
/=	Divide y guarda (queda float).	<code>x = 10 x /= 4 print(x)</code>	2.5
//=	División entera y guarda.	<code>x = 10 x //= 3 print(x)</code>	3
%=	Residuo y guarda.	<code>x = 10 x %= 3 print(x)</code>	1
**=	Potencia y guarda.	<code>x = 3 x **= 2 print(x)</code>	9
Contador	Cuenta repeticiones.	<code>c = 0 for _ in range(4): c += 1 print(c)</code>	4
Acumulador	Va sumando valores.	<code>total = 0 for n in [2, 4, 6]: total += n print(total)</code>	12
x++ no existe	Es un error de sintaxis.	<code>x = 1 x++</code>	SyntaxError: invalid syntax

Módulo math

El módulo math trae funciones matemáticas. Se importa con `import math` y se usa como `math.función()`.

RECUERDA `math` = la calculadora científica de Python. Siempre: `import math` primero.

DATOS DE EJEMPLO

```
import math
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>math.sqrt()</code>	Raíz cuadrada.	<code>math.sqrt(16)</code>	4.0
<code>math.pi</code>	El número pi.	<code>math.pi</code>	3.141592653589793
<code>math.e</code>	El número e.	<code>math.e</code>	2.718281828459045
<code>math.floor()</code>	Redondea hacia abajo.	<code>math.floor(3.7)</code>	3
<code>math.ceil()</code>	Redondea hacia arriba.	<code>math.ceil(3.2)</code>	4
<code>math.trunc()</code>	Quita los decimales.	<code>math.trunc(-3.7)</code>	-3
<code>math.factorial()</code>	Factorial: $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$.	<code>math.factorial(5)</code>	120
<code>math.pow()</code>	Potencia (devuelve float).	<code>math.pow(2, 3)</code>	8.0
<code>math.gcd()</code>	Máximo común divisor.	<code>math.gcd(12, 18)</code>	6
<code>math.hypot()</code>	Hipotenusa del triángulo.	<code>math.hypot(3, 4)</code>	5.0
<code>math.log10()</code>	Logaritmo en base 10.	<code>math.log10(1000)</code>	3.0
<code>math.sin()</code>	Seno (en radianes).	<code>math.sin(math.pi / 2)</code>	1.0
<code>math.degrees()</code>	De radianes a grados.	<code>math.degrees(math.pi)</code>	180.0
<code>math.comb()</code>	Combinaciones de 5 tomadas de 2 en 2.	<code>math.comb(5, 2)</code>	10

Entrada y salida de datos

La salida se hace con `print()` y la entrada con `input()`. `input()` siempre devuelve texto (`str`); si necesitas un número, conviértelo con `int()` o `float()`.

RECUERDA `print` = Python habla. `input` = Python escucha (y lo que escucha llega como texto).

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>print()</code>	Muestra texto o valores.	<code>print("Hola")</code>	Hola
Varios valores	Los separa con un espacio.	<code>print(1, 2, 3)</code>	1 2 3
sep=	Cambia el separador.	<code>print("a", "b", "c", sep="-")</code>	a-b-c
end=	Cambia el final (por defecto salta de línea).	<code>print("Hola", end=" ")</code> <code>print("GM")</code>	Hola GM
Línea vacía	<code>print()</code> sin nada deja un renglón libre.	<code>print("a")</code> <code>print()</code> <code>print("b")</code>	a b
Con variables	Mezcla texto y variables.	<code>nombre = "Ana"</code> <code>print("Hola", nombre)</code>	Hola Ana
input()	Pide un texto al usuario.	<code>nombre = input("Nombre: ")</code> <code>print("Hola", nombre)</code>	Nombre: Luis Hola Luis

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>input() es str</code>	Aunque escriban un número.	<code>x = input("Edad: ") print(type(x))</code>	Edad: 19 <class 'str'>
<code>input() + int()</code>	Convierte a entero.	<code>edad = int(input("Edad: ")) print(edad + 1)</code>	Edad: 19 20
<code>input() + float()</code>	Convierte a decimal.	<code>precio = float(input("Precio: ")) print(precio * 2)</code>	Precio: 2.5 5.0
<code>split()</code>	Dos datos en una sola línea.	<code>a, b = input("A B: ").split() print(a, b)</code>	A B: 3 4 3 4
<code>map(int, ...)</code>	Dos enteros en una línea.	<code>a, b = map(int, input().split()) print(a + b)</code>	3 4 7

TEMA 19

Conversiones

Convertir (casting) es pasar un valor de un tipo a otro con `int()`, `float()`, `str()`, `bool()`, `list()`, etc. Si el valor no se puede convertir, Python lanza un `ValueError`.

RECUERDA `int()` corta, `float()` agrega .0, `str()` lo vuelve texto. Si no se puede convertir, `ValueError`.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>int()</code>	De texto a entero.	<code>int("25")</code>	25
<code>int() corta</code>	De decimal a entero: corta, no redondea.	<code>int(7.9)</code>	7
<code>float()</code>	De texto a decimal.	<code>float("3.5")</code>	3.5
<code>float()</code>	De entero a decimal.	<code>float(4)</code>	4.0
<code>str()</code>	A texto.	<code>str(100)</code>	'100'
<code>bool()</code>	Cero y vacío son False.	<code>print(bool(0), bool("hola"), bool(""))</code>	False True False
<code>int() falla</code>	El texto no es un número.	<code>int("abc")</code>	ValueError: invalid literal for int() with base 10: 'abc'
<code>int() de decimal en texto</code>	Primero <code>float()</code> , luego <code>int()</code> .	<code>int(float("3.5"))</code>	3
<code>list()</code>	Texto a lista de letras.	<code>list("abc")</code>	['a', 'b', 'c']
<code>tuple()</code>	Lista a tupla.	<code>tuple([1, 2])</code>	(1, 2)
<code>set()</code>	Lista a conjunto (sin repetidos).	<code>set([1, 1, 2])</code>	{1, 2}
<code>ord() / chr()</code>	Letra ↔ número de código.	<code>print(ord("A"), chr(66))</code>	65 B
<code>int(texto, base)</code>	Binario a decimal.	<code>int("101", 2)</code>	5
<code>bin()</code>	Decimal a binario.	<code>bin(5)</code>	'0b101'

Errores comunes

Hay tres tipos: de sintaxis (código mal escrito), de ejecución (falla al correr) y lógicos (corre, pero da un resultado incorrecto). Lee el mensaje del error: casi siempre te dice qué pasó.

RECUERDA Lee el error de abajo hacia arriba: la última línea dice qué falló y el número de línea dice dónde.

ERROR	DESCRIPCIÓN	EJEMPLO	RESULTADO
SyntaxError	Falta cerrar un paréntesis.	<code>print("hola"</code>	SyntaxError: '(' was never closed
IndentationError	Falta la indentación.	<code>if True: print(1)</code>	IndentationError: expected an indented block after 'if' statement on line 1
NameError	Usas una variable que no existe.	<code>print(x)</code>	NameError: name 'x' is not defined
TypeError	Mezclas tipos que no se pueden juntar.	<code>"5" + 3</code>	TypeError: can only concatenate str (not "int") to str
ValueError	Valor con tipo correcto pero inválido.	<code>int("abc")</code>	ValueError: invalid literal for int() with base 10: 'abc'
ZeroDivisionError	Dividir entre cero.	<code>5 / 0</code>	ZeroDivisionError: division by zero
IndexError	Posición que no existe.	<code>[1, 2, 3][5]</code>	IndexError: list index out of range
KeyError	Clave que no existe en el diccionario.	<code>{"a": 1}["b"]</code>	KeyError: 'b'
AttributeError	Método que ese tipo no tiene.	<code>"hola".push()</code>	AttributeError: 'str' object has no attribute 'push'
Error lógico	Sin error, pero mal: falta el paréntesis del promedio.	<code>print(6 + 8 + 10 / 3)</code>	17.333333333333332

Format (f-strings)

Formatear es insertar valores dentro de un texto con control de decimales, ancho y alineación. Lo más cómodo son las f-strings: `f"texto {variable}"`. Error común: olvidar la `f` al inicio.

RECUERDA La `f` al inicio es el «interruptor»: sin ella, las llaves se imprimen tal cual.

DATOS DE EJEMPLO

```
nombre = "Ana"
nota = 9.456
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>f"{x}"</code>	Inserta una variable.	<code>print(f"Hola {nombre}")</code>	Hola Ana
Expresión	Dentro de las llaves se puede calcular.	<code>print(f"Doble: {nota * 2}")</code>	Doble: 18.912
<code>:.2f</code>	Dos decimales.	<code>print(f"{nota:.2f}")</code>	9.46
<code>:.0f</code>	Sin decimales.	<code>print(f"{nota:.0f}")</code>	9
<code>>6</code>	Alinea a la derecha en 6 espacios.	<code>print(f"[{nombre:>6}]")</code>	[Ana]
<code><6</code>	Alinea a la izquierda.	<code>print(f"[{nombre:<6}]")</code>	[Ana]

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>:^7</code>	Centra el texto.	<code>print(f"[{nombre:^7}]")</code>	[Ana]
<code>:03d</code>	Rellena con ceros.	<code>print(f"{7:03d}")</code>	007
<code>:,</code>	Separador de miles.	<code>print(f"{1234567:,}")</code>	1,234,567
<code>:.1%</code>	Porcentaje.	<code>print(f"{0.256:.1%}")</code>	25.6%
<code>{x=}</code>	Muestra nombre y valor (útil para depurar).	<code>print(f"{nota=}")</code>	nota=9.456
<code>.format()</code>	Forma antigua con llaves vacías.	<code>print("Hola {}, nota {}".format(nombre, 9))</code>	Hola Ana, nota 9
<code>.format()</code> con índices	Elige el orden con números.	<code>print("{1} {0}".format("a", "b"))</code>	b a
<code>% antiguo</code>	Estilo heredado de C.	<code>print("%s tiene %.1f" % (nombre, nota))</code>	Ana tiene 9.5
Sin la f	Error común: se imprime la llave tal cual.	<code>print("Hola {nombre}")</code>	Hola {nombre}
<code>:b / :x</code>	Binario y hexadecimal.	<code>print(f"{10:b} {255:x}")</code>	1010 ff

TEMA 22

Decimales y redondeo

Los float guardan aproximaciones, por eso $0.1 + 0.2$ no da exactamente 0.3. Para mostrar menos decimales usa `round()` o `format():.2f`. Ojo: `round()` redondea el .5 al par más cercano.

RECUERDA Para mostrar: `f"{x:.2f}"`. Para calcular con cuidado: `round(x, 2)`.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>round(x, n)</code>	Redondea a n decimales.	<code>round(3.14159, 2)</code>	3.14
<code>round(x)</code>	Redondea a entero.	<code>round(7.4)</code>	7
<code>round()</code> al par	El .5 va al entero par más cercano.	<code>print(round(2.5), round(3.5))</code>	2 4
<code>round(x, -2)</code>	Redondea a centenas.	<code>round(1234, -2)</code>	1200
<code>int()</code>	Corta los decimales.	<code>print(int(3.9), int(-3.9))</code>	3 -3
<code>math.floor()</code>	Siempre hacia abajo.	<code>import math print(math.floor(-3.1))</code>	-4
<code>math.ceil()</code>	Siempre hacia arriba.	<code>import math print(math.ceil(3.1))</code>	4
Error de float	Aproximación binaria.	<code>0.1 + 0.2</code>	0.30000000000000004
Arreglarlo	Redondea el resultado.	<code>round(0.1 + 0.2, 2)</code>	0.3
<code>math.isclose()</code>	Compara decimales con tolerancia.	<code>import math math.isclose(0.1 + 0.2, 0.3)</code>	True
Muchos decimales	Divisiones infinitas se cortan.	<code>10 / 3</code>	3.3333333333333335
Formato	Muestra 3 decimales sin cambiar el valor.	<code>print(f"{10 / 3:.3f}")</code>	3.333

Cadenas de texto (str)

Cómo guardar, recorrer, comparar y transformar texto.

TEMA 23

Tipo de dato: string

Un string (str) es texto entre comillas simples, dobles o triples. Es una secuencia de caracteres ordenados y es inmutable: no se puede cambiar una letra, solo crear un texto nuevo.

RECUERDA String = collar de letras. Puedes mirar cada cuenta, pero para cambiar una tienes que armar otro collar.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Comillas dobles	Texto normal.	<code>print("Hola")</code>	Hola
Comillas simples	Igual de válidas.	<code>print('Hola')</code>	Hola
Comillas mixtas	Una clase de comilla dentro de la otra.	<code>print("It's GM")</code>	It's GM
Triples	Texto en varias líneas.	<code>print("""línea 1 línea 2""")</code>	línea 1 línea 2
str vacío	Texto sin caracteres.	<code>len("")</code>	0
Inmutable	No se puede cambiar una letra.	<code>s = "hola" s[0] = "H"</code>	<code>TypeError: 'str' object does not support item assignment</code>
Crear uno nuevo	Se arma otro texto con la letra cambiada.	<code>s = "hola" s = "H" + s[1:] print(s)</code>	Hola
Recorrer	for pasa letra por letra.	<code>for letra in "GM": print(letra)</code>	G M
str()	Convierte un número en texto.	<code>str(2026) + "!"</code>	'2026!'
Número ≠ texto	"5" no es 5.	<code>"5" == 5</code>	False

TEMA 24

Index de str

Cada letra tiene una posición (índice). Se empieza a contar desde 0. Los índices negativos cuentan desde el final: -1 es la última letra. Salirse del rango da IndexError.

RECUERDA Los índices empiezan en 0, como los pisos de un edificio: la planta baja es el 0. Y -1 es «el último».

DATOS DE EJEMPLO

`s = "PYTHON"`

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>s[0]</code>	Primera letra.	<code>s[0]</code>	'P'
<code>s[1]</code>	Segunda letra.	<code>s[1]</code>	'Y'
<code>s[5]</code>	Última (la 6.ª) letra.	<code>s[5]</code>	'N'
<code>s[-1]</code>	Última letra con índice negativo.	<code>s[-1]</code>	'N'
<code>s[-2]</code>	Penúltima letra.	<code>s[-2]</code>	'O'
<code>s[len(s) - 1]</code>	Última letra con len().	<code>s[len(s) - 1]</code>	'N'

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>s[6]</code>	Fuera del rango.	<code>s[6]</code>	<code>IndexError: string index out of range</code>
<code>s.index()</code>	Posición de una letra.	<code>s.index("H")</code>	3
Letra por letra	Recorre con índices.	<pre>for i in range(3): print(i, s[i])</pre>	0 P 1 Y 2 T
<code>enumerate()</code>	Índice y letra a la vez.	<pre>for i, c in enumerate("GM"): print(i, c)</pre>	0 G 1 M

TEMA 25

Slicing de str

Slicing es cortar un pedazo del texto con `[inicio:fin:paso]`. El inicio sí se incluye y el fin NO. Si omites un valor, Python usa el principio, el final o paso 1.

RECUERDA `[inicio:fin]` — el fin es la «puerta de salida»: llegas hasta ella, pero no la cruzas.

DATOS DE EJEMPLO
`s = "PYTHON"`

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>s[0:3]</code>	De la posición 0 a la 2.	<code>s[0:3]</code>	'PYT'
<code>s[2:]</code>	Desde la posición 2 hasta el final.	<code>s[2:]</code>	'THON'
<code>s[:2]</code>	Desde el inicio hasta la posición 1.	<code>s[:2]</code>	'PY'
<code>s[-3:]</code>	Las últimas 3 letras.	<code>s[-3:]</code>	'HON'
<code>s[::2]</code>	De 2 en 2.	<code>s[::2]</code>	'PTO'
<code>s[1:5:2]</code>	Inicio, fin y paso.	<code>s[1:5:2]</code>	'YH'
<code>s[::-1]</code>	Texto al revés.	<code>s[::-1]</code>	'NOHTYP'
<code>s[:]</code>	Copia completa.	<code>s[:]</code>	'PYTHON'
<code>s[10:]</code>	Pasarse no da error: texto vacío.	<code>s[10:]</code>	''
Palíndromo	¿Se lee igual al revés?	<pre>p = "oso" print(p == p[::-1])</pre>	True

TEMA 26

Operaciones de comparación

Los textos se comparan con `==`, `!=`, `<`, `>`, `<=`, `>=`. La comparación es letra por letra según su código: las mayúsculas valen menos que las minúsculas, por eso «A» < «a».

RECUERDA «A» y «a» son letras distintas para Python. Para comparar sin importar mayúsculas, pasa todo a minúsculas.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>==</code>	¿Son idénticos?	<code>"hola" == "hola"</code>	True
Mayúsculas	A y a son distintos.	<code>"Hola" == "hola"</code>	False
Sin importar mayúsculas	Pasa ambos a minúsculas.	<code>"Hola".lower() == "hola".lower()</code>	True
<code>!=</code>	¿Son distintos?	<code>"a" != "b"</code>	True
<code><</code>	Orden alfabético.	<code>"casa" < "perro"</code>	True

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Mayúscula antes	Las mayúsculas van antes que las minúsculas.	<code>"Zebra" < "abeja"</code>	True
ord()	Código de cada letra.	<code>print(ord("A"), ord("a"))</code>	65 97
Números como texto	Se comparan como texto, no como número.	<code>"10" < "9"</code>	True
Espacios	Un espacio extra cambia todo.	<code>"casa" == "casa "</code>	False
strip() antes	Quita espacios antes de comparar.	<code>"casa ".strip() == "casa"</code>	True

TEMA 27

Secuencias de escape

Una secuencia de escape empieza con `\` y representa un carácter especial dentro de un texto: salto de línea, tabulador, comillas, etc. Con `r"..."` (raw) las barras se escriben tal cual.

RECUERDA La barra invertida `\` es el «interruptor especial»: cambia el significado de la letra que viene después.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>\n</code>	Salto de línea.	<code>print("Hola\nMundo")</code>	Hola Mundo
<code>\t</code>	Tabulador (espacio grande).	<code>print("a\tb")</code>	a b
<code>\"</code>	Comilla doble dentro del texto.	<code>print("Dijo \"hola\"")</code>	Dijo "hola"
<code>\'</code>	Comilla simple dentro del texto.	<code>print('It\'s GM')</code>	It's GM
<code>\\</code>	Una barra invertida.	<code>print("C:\\datos")</code>	C:\datos
<code>r"..."</code>	Texto crudo: la barra se queda.	<code>print(r"C:\nuevo")</code>	C:\nuevo
<code>\u</code>	Carácter Unicode.	<code>print("\u00e1")</code>	á
<code>\r</code>	Regresa al inicio de la línea.	<code>print("abc\rX")</code>	Xbc (en terminal)
<code>len()</code> y <code>\n</code>	El salto cuenta como 1 carácter.	<code>len("a\nb")</code>	3

TEMA 28

Tabulador

`\t` inserta un tabulador: salta hasta la siguiente columna (cada 8 espacios por defecto). Sirve para alinear texto en columnas sin calcular espacios.

RECUERDA `\t` es la tecla Tab dentro de un texto: alinea columnas como en una tabla.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>\t</code>	Un tabulador entre palabras.	<code>print("Nombre\tNota")</code>	Nombre Nota
Tabla simple	Alinea columnas.	<code>print("Ana\t9")</code> <code>print("Luis\t8")</code>	Ana 9 Luis 8
Varias columnas	Con más de un <code>\t</code> .	<code>print("A\tB\tC")</code>	A B C
<code>sep="\t"</code>	<code>print</code> separa con tabulador.	<code>print("A", "B", "C", sep="\t")</code>	A B C
<code>expandtabs()</code>	Cambia el ancho del tab.	<code>"a\tb".expandtabs(4)</code>	'a b'
<code>end="\t"</code>	Termina con tab en vez de salto.	<code>print("x", end="\t")</code> <code>print("y")</code>	x y
Encabezado y fila	Tabla con <code>\n</code> y <code>\t</code> .	<code>print("Nombre\tNota\nAna\t9")</code>	Nombre Nota Ana 9

Operadores en str

Con texto solo funcionan + (concatenar: unir) y * (repetir un texto n veces). No existen - ni / para strings. Para unir texto con un número hay que convertirlo con str() o usar f-strings.

RECUERDA + pega textos como si usaras cinta. * fotocopia el texto varias veces.

OPERADOR	DESCRIPCIÓN	EJEMPLO	RESULTADO
+	Concatena (une).	"Hola " + "GM"	'Hola GM'
*	Repite el texto.	"ja" * 3	'jajaja'
Línea decorativa	Una raya con *.	print("=" * 10)	=====
+=	Va agregando al final.	s = "a" s += "b" s += "c" print(s)	abc
texto + número	Da error: hay que convertir.	"Edad: " + 19	TypeError: can only concatenate str (not "int") to str
Convertir con str()	Se arregla con str().	"Edad: " + str(19)	'Edad: 19'
f-string	Más cómodo: une sin convertir.	edad = 19 print(f"Edad: {edad}")	Edad: 19
Resta	No existe para texto.	"abc" - "a"	TypeError: unsupported operand type(s) for -: 'str' and 'str'
Comparación	Sí se puede comparar.	"a" < "b"	True

Operador in

in revisa si un texto (o una letra) está dentro de otro. Devuelve True o False. not in pregunta lo contrario. También funciona con listas, tuplas y diccionarios.

RECUERDA in = «¿está dentro?». Como buscar una palabra con Ctrl + F.

USO	DESCRIPCIÓN	EJEMPLO	RESULTADO
letra in texto	¿Está esa letra?	"a" in "casa"	True
palabra in texto	¿Está esa parte?	"ca" in "casa"	True
No está	Devuelve False.	"xyz" in "casa"	False
Mayúsculas	Distingue mayúsculas.	"A" in "casa"	False
not in	¿NO está dentro?	"z" not in "casa"	True
En una lista	Funciona con elementos de lista.	3 in [1, 2, 3]	True
En un diccionario	Busca entre las claves.	"a" in {"a": 1}	True
¿Es vocal?	Comprobación clásica.	print("e" in "aeiou")	True
En un if	Úsalo para decidir.	if "@" in "ana@mail.com": print("correo")	correo
Texto vacío	Siempre está en cualquier texto.	"" in "hola"	True

Funciones de str (formato)

Los strings tienen métodos propios que se llaman con un punto: texto.método(). No modifican el original: devuelven un texto nuevo. Aquí van los de limpieza y alineación.

RECUERDA El texto original nunca cambia. Guarda el resultado en una variable si lo quieres usar.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>strip()</code>	Quita espacios de ambos lados.	<code>" hola ".strip()</code>	<code>'hola'</code>
<code>lstrip()</code>	Quita espacios a la izquierda.	<code>" hola ".lstrip()</code>	<code>'hola '</code>
<code>rstrip()</code>	Quita espacios a la derecha.	<code>" hola ".rstrip()</code>	<code>' hola'</code>
<code>strip(chars)</code>	Quita esos caracteres de los bordes.	<code>"--hola--".strip("-")</code>	<code>'hola'</code>
<code>center()</code>	Centra en un ancho dado.	<code>"GM".center(8, "*")</code>	<code>'***GM***'</code>
<code>ljust()</code>	Alinea a la izquierda.	<code>"GM".ljust(6, ".")</code>	<code>'GM....'</code>
<code>rjust()</code>	Alinea a la derecha.	<code>"GM".rjust(6, ".")</code>	<code>'...GM'</code>
<code>zfill()</code>	Rellena con ceros a la izquierda.	<code>"7".zfill(3)</code>	<code>'007'</code>
<code>join()</code>	Une una lista con un separador.	<code>"-".join(["a", "b", "c"])</code>	<code>'a-b-c'</code>
<code>partition()</code>	Separa en 3 partes por un texto.	<code>"x=5".partition("=")</code>	<code>('x', '=', '5')</code>
No modifica	El original queda igual.	<pre>s = "hola" s.upper() print(s)</pre>	<code>hola</code>
Encadenar	Varios métodos seguidos.	<code>" HOLA ".strip().lower()</code>	<code>'hola'</code>

Longitud de cadena

`len()` devuelve cuántos caracteres tiene un texto. Los espacios, signos y saltos de línea también cuentan. El último índice siempre es `len(texto) - 1`.

RECUERDA `len()` cuenta todas las cuentas del collar, incluso las invisibles (espacios).

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>len("hola")</code>	Cantidad de caracteres.	<code>len("hola")</code>	4
Texto vacío	Tiene longitud 0.	<code>len("")</code>	0
Espacios	Los espacios cuentan.	<code>len("a b")</code>	3
Signos y tildes	Cuentan como 1 cada uno.	<code>len(";Hola!")</code>	6
Número como texto	Convierte primero con <code>str()</code> .	<code>len(str(12345))</code>	5
Último índice	Siempre es <code>len - 1</code> .	<pre>t = "GM" print(len(t) - 1)</pre>	1
Salto de línea	<code>\n</code> cuenta como 1.	<code>len("a\nb")</code>	3
len en if	Validar una contraseña corta.	<pre>clave = "abc" print(len(clave) >= 8)</pre>	False
Contar palabras	<code>len()</code> sobre <code>split()</code> .	<code>len("hola mundo GM".split())</code>	3

Contar con count()

texto.count(parte) cuenta cuántas veces aparece una parte dentro del texto, sin traslapes. Distingue mayúsculas y acepta posiciones de inicio y fin.

RECUERDA count() es el contador de cuántas veces aparece algo. Para ignorar mayúsculas: pasa todo a minúsculas antes.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
count("a")	Cuántas veces aparece una letra.	"banana".count("a")	3
count("an")	También busca partes.	"banana".count("an")	2
Sin traslape	Cuenta sin repetir letras.	"aaaa".count("aa")	2
Con inicio	Empieza a contar desde una posición.	"banana".count("a", 2)	2
Mayúsculas	A y a son distintas.	"Aa".count("a")	1
Sin importar mayúsculas	Pasa a minúsculas.	"Hola mundo".lower().count("o")	2
No aparece	Devuelve 0.	"hola".count("z")	0
Contar vocales	Suma el conteo de cada vocal.	print(sum("murcielago".count(v) for v in "aeiou"))	5
Texto vacío	Cuenta posiciones entre letras.	"hola".count("")	5

Reemplazar con replace()

texto.replace(viejo, nuevo) devuelve un texto nuevo cambiando todas las apariciones. Con un tercer valor limita cuántas veces cambia. No modifica el original.

RECUERDA replace() es «Buscar y reemplazar» de Word. Guarda el resultado, porque el original no cambia.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
replace(a, b)	Cambia todas las apariciones.	"banana".replace("a", "o")	'bonono'
Con límite	Cambia solo las 2 primeras.	"banana".replace("a", "o", 2)	'bonona'
Borrar	Reemplaza por vacío.	"a-b-c".replace("-", "")	'abc'
Quitar espacios	Todos los espacios.	"a b c".replace(" ", "")	'abc'
Distingue mayúsculas	No cambia la A.	"Ana".replace("a", "o")	'Ano'
Palabras	También sirve con palabras.	"hola mundo".replace("mundo", "GM")	'hola GM'
Encadenar	Varios cambios seguidos.	"a,b;c".replace(", ", ";").replace(";", " ")	'a b c'
No modifica	El original queda igual.	s = "casa" s.replace("a", "o") print(s)	casa
Guardar el cambio	Reasigna la variable.	s = "casa" s = s.replace("a", "o") print(s)	coso

Cambiar mayúsculas y minúsculas

Estos métodos devuelven una versión del texto con otro uso de mayúsculas. Son muy útiles para comparar sin importar cómo escribió el usuario.

RECUERDA `lower()` para comparar, `title()` para nombres, `upper()` para gritar.

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>upper()</code>	Todo en MAYÚSCULAS.	<code>"hola gm".upper()</code>	<code>'HOLA GM'</code>
<code>lower()</code>	Todo en minúsculas.	<code>"HOLA GM".lower()</code>	<code>'hola gm'</code>
<code>capitalize()</code>	Solo la primera letra en mayúscula.	<code>"hola mundo".capitalize()</code>	<code>'Hola mundo'</code>
<code>title()</code>	Cada palabra con mayúscula.	<code>"hola mundo gm".title()</code>	<code>'Hola Mundo Gm'</code>
<code>swapcase()</code>	Invierte mayúsculas y minúsculas.	<code>"HoLa".swapcase()</code>	<code>'h0lA'</code>
<code>casefold()</code>	Minúsculas agresivas (para comparar).	<code>"Straße".casefold()</code>	<code>'strasse'</code>
Comparar	Ignora cómo escribió el usuario.	<code>r = "SI"</code> <code>print(r.lower() == "si")</code>	<code>True</code>
Nombre propio	Arregla un nombre mal escrito.	<code>"aNa pÉREz".title()</code>	<code>'Ana Pérez'</code>

Buscar en un texto

`find()` y `index()` dicen en qué posición aparece una parte. `find()` devuelve -1 si no la encuentra; `index()` da error. `startswith()` y `endswith()` revisan el inicio y el final.

RECUERDA `find()` es el amable (-1 si no hay). `index()` es el estricto (`ValueError` si no hay).

DATOS DE EJEMPLO
`s = "banana"`

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>find()</code>	Posición de la primera aparición.	<code>s.find("a")</code>	<code>1</code>
<code>find()</code> no encontrado	Devuelve -1.	<code>s.find("z")</code>	<code>-1</code>
<code>find(x, inicio)</code>	Busca desde una posición.	<code>s.find("a", 2)</code>	<code>3</code>
<code>rfind()</code>	Busca desde el final.	<code>s.rfind("a")</code>	<code>5</code>
<code>index()</code>	Igual que <code>find</code> , pero con error.	<code>s.index("n")</code>	<code>2</code>
<code>index()</code> no encontrado	Da <code>ValueError</code> .	<code>s.index("z")</code>	<code>ValueError: substring not found</code>
<code>startswith()</code>	¿Empieza con...?	<code>s.startswith("ban")</code>	<code>True</code>
<code>endswith()</code>	¿Termina con...?	<code>s.endswith("na")</code>	<code>True</code>
<code>endswith(tupla)</code>	¿Termina con alguna de estas?	<code>"foto.png".endswith((".png", ".jpg"))</code>	<code>True</code>
<code>in</code>	Solo saber si está o no.	<code>"nan" in s</code>	<code>True</code>

Funciones que comienzan con is...

Los métodos is... responden con True o False sobre qué tipo de caracteres tiene el texto. Son perfectos para validar lo que escribe el usuario antes de convertirlo.

RECUERDA Siempre empiezan con «is» y siempre responden Sí o No (True o False).

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>isdigit()</code>	¿Son solo dígitos?	<code>"123".isdigit()</code>	True
<code>isdigit()</code> con punto	El punto no es dígito.	<code>"12.5".isdigit()</code>	False
<code>isalpha()</code>	¿Son solo letras?	<code>"hola".isalpha()</code>	True
<code>isalpha()</code> con número	Un número lo arruina.	<code>"hola1".isalpha()</code>	False
<code>isalnum()</code>	¿Letras y/o números?	<code>"abc123".isalnum()</code>	True
<code>isupper()</code>	¿Todo en mayúsculas?	<code>"ABC".isupper()</code>	True
<code>islower()</code>	¿Todo en minúsculas?	<code>"abc".islower()</code>	True
<code>isspace()</code>	¿Solo espacios?	<code>" ".isspace()</code>	True
<code>istitle()</code>	¿Cada palabra con mayúscula?	<code>"Hola Mundo".istitle()</code>	True
<code>isidentifier()</code>	¿Sirve como nombre de variable?	<code>"mi_var".isidentifier()</code>	True
Texto vacío	Todos dan False con "".	<code>"".isdigit()</code>	False
Validar antes de <code>int()</code>	Evita el ValueError.	<pre>x = "42" if x.isdigit(): print(int(x) + 1)</pre>	43

Listas y funciones generales

Guardar varios datos juntos y las herramientas para trabajar con ellos.

TEMA 38

Tipo de dato: lista

Una lista guarda varios valores en orden, entre corchetes `[]`. Es mutable: se puede cambiar, agregar y borrar elementos. Admite tipos mezclados y se indexa igual que un string (desde 0).

RECUERDA Lista = mochila con compartimentos numerados desde 0. Puedes meter y sacar cosas cuando quieras.

DATOS DE EJEMPLO

```
frutas = ["pera", "uva", "kiwi"]
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>[]</code>	Crear una lista.	<code>[1, 2, 3]</code>	<code>[1, 2, 3]</code>
Lista vacía	Sin elementos.	<code>len([])</code>	0
Tipos mezclados	Puede tener de todo.	<code>[1, "a", 2.5, True]</code>	<code>[1, 'a', 2.5, True]</code>
<code>lista[0]</code>	Primer elemento.	<code>print(frutas[0])</code>	pera
<code>lista[-1]</code>	Último elemento.	<code>print(frutas[-1])</code>	kiwi
Modificar	Cambia un elemento (mutable).	<code>frutas[1] = "mango"</code> <code>print(frutas)</code>	<code>['pera', 'mango', 'kiwi']</code>
<code>len()</code>	Cantidad de elementos.	<code>len(frutas)</code>	3
<code>list()</code>	Crea una lista desde otra cosa.	<code>list(range(5))</code>	<code>[0, 1, 2, 3, 4]</code>
<code>in</code>	¿Está el elemento?	<code>"uva" in frutas</code>	True
Recorrer	for pasa elemento por elemento.	<code>for f in frutas:</code> <code>print(f)</code>	pera uva kiwi
IndexError	Posición que no existe.	<code>frutas[5]</code>	IndexError: list index out of range
Misma lista	b = a NO copia: ambas son la misma lista.	<code>a = [1, 2]</code> <code>b = a</code> <code>b.append(3)</code> <code>print(a)</code>	<code>[1, 2, 3]</code>

TEMA 39

Slicing de listas

Funciona igual que en strings: `lista[inicio:fin:paso]`. El fin no se incluye. El resultado siempre es una lista nueva. Además se puede reemplazar o borrar un pedazo.

RECUERDA Mismo truco que con los textos: `[inicio:fin]` — el fin es la puerta que no cruzas.

DATOS DE EJEMPLO

```
nums = [10, 20, 30, 40, 50]
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>[1:3]</code>	Posiciones 1 y 2.	<code>nums[1:3]</code>	<code>[20, 30]</code>
<code>[:2]</code>	Los 2 primeros.	<code>nums[:2]</code>	<code>[10, 20]</code>
<code>[2:]</code>	Desde la posición 2.	<code>nums[2:]</code>	<code>[30, 40, 50]</code>

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>[-2:]</code>	Los 2 últimos.	<code>nums[-2:]</code>	<code>[40, 50]</code>
<code>::2</code>	De 2 en 2.	<code>nums::2</code>	<code>[10, 30, 50]</code>
<code>[1:4:2]</code>	Inicio, fin y paso.	<code>nums[1:4:2]</code>	<code>[20, 40]</code>
<code>::-1</code>	Lista al revés.	<code>nums::-1</code>	<code>[50, 40, 30, 20, 10]</code>
<code>[:]</code>	Copia de la lista.	<code>copia = nums[:]</code> <code>copia.append(0)</code> <code>print(nums)</code>	<code>[10, 20, 30, 40, 50]</code>
Reemplazar pedazo	Cambia varias posiciones.	<code>nums[1:3] = [0]</code> <code>print(nums)</code>	<code>[10, 0, 40, 50]</code>
Borrar pedazo	Quita las 2 primeras.	<code>del nums[:2]</code> <code>print(nums)</code>	<code>[30, 40, 50]</code>
Fuera de rango	No da error: lista vacía.	<code>nums[10:]</code>	<code>[]</code>

TEMA 40

Operadores en listas

Con listas se usan + (unir listas), * (repetir), in / not in (pertenencia) y los de comparación. No se puede sumar una lista con un número.

RECUERDA + junta dos listas como unir dos trenes. * repite el mismo tren varias veces.

OPERADOR	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>+</code>	Une dos listas.	<code>[1, 2] + [3, 4]</code>	<code>[1, 2, 3, 4]</code>
<code>*</code>	Repite la lista.	<code>[0, 1] * 3</code>	<code>[0, 1, 0, 1, 0, 1]</code>
<code>[0] * n</code>	Lista de n ceros.	<code>[0] * 5</code>	<code>[0, 0, 0, 0, 0]</code>
<code>+=</code>	Agrega elementos al final.	<code>l = [1]</code> <code>l += [2, 3]</code> <code>print(l)</code>	<code>[1, 2, 3]</code>
<code>in</code>	¿Está el elemento?	<code>3 in [1, 2, 3]</code>	<code>True</code>
<code>not in</code>	¿NO está?	<code>9 not in [1, 2, 3]</code>	<code>True</code>
<code>==</code>	¿Mismos elementos y mismo orden?	<code>[1, 2] == [1, 2]</code>	<code>True</code>
<code><</code>	Compara elemento por elemento.	<code>[1, 2] < [1, 3]</code>	<code>True</code>
+ con número	Da error: hay que usar [3].	<code>[1, 2] + 3</code>	<code>TypeError: can only concatenate list (not "int") to list</code>
Mismo contenido, otro orden	No son iguales.	<code>[1, 2] == [2, 1]</code>	<code>False</code>

Métodos de listas

Los métodos de lista modifican la lista original (casi todos devuelven None). Los más usados: append, insert, remove, pop, sort, reverse, index y count.

RECUERDA append() agrega al final. insert() agrega donde tú digas. pop() saca y te lo devuelve.

DATOS DE EJEMPLO
nums = [4, 1, 3]

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
append(x)	Agrega al final.	nums.append(5) nums	[4, 1, 3, 5]
extend(lista)	Agrega varios al final.	nums.extend([7, 8]) nums	[4, 1, 3, 7, 8]
insert(i, x)	Agrega en la posición i.	nums.insert(1, 9) nums	[4, 9, 1, 3]
remove(x)	Borra la primera aparición de x.	nums.remove(1) nums	[4, 3]
pop()	Saca y devuelve el último.	x = nums.pop() print(x, nums)	3 [4, 1]
pop(i)	Saca el de la posición i.	x = nums.pop(0) print(x, nums)	4 [1, 3]
sort()	Ordena de menor a mayor.	nums.sort() nums	[1, 3, 4]
sort(reverse=True)	Ordena de mayor a menor.	nums.sort(reverse=True) nums	[4, 3, 1]
reverse()	Invierte el orden.	nums.reverse() nums	[3, 1, 4]
index(x)	Posición de x.	nums.index(3)	2
count(x)	Cuántas veces está x.	nums.count(1)	1
copy()	Copia independiente.	c = nums.copy() c.append(0) print(nums, c)	[4, 1, 3] [4, 1, 3, 0]
clear()	Vacía la lista.	nums.clear() nums	[]
del lista[i]	Borra por posición.	del nums[0] nums	[1, 3]
sorted(lista)	Devuelve una lista ordenada nueva.	print(sorted(nums), nums)	[1, 3, 4] [4, 1, 3]
sort() devuelve None	Ojo: no guardes su resultado.	print(nums.sort())	None
remove() inexistente	Da ValueError.	nums.remove(9)	ValueError: list.remove(x): x not in list

Métodos split() y join()

split() corta un texto en una lista de pedazos usando un separador (por defecto, los espacios). join() hace lo contrario: une una lista de textos con un separador.

RECUERDA split() = tijeras: corta el texto en pedazos. join() = pegamento: une los pedazos.

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
split()	Corta por espacios.	"hola mundo GM".split()	['hola', 'mundo', 'GM']
split(",")	Corta por un separador.	"1,2,3".split(",")	['1', '2', '3']
split(sep, n)	Máximo n cortes.	"a-b-c-d".split("-", 1)	['a', 'b-c-d']
Espacios repetidos	split() sin argumento los ignora.	" hola mundo ".split()	['hola', 'mundo']
splitlines()	Corta por saltos de línea.	"uno\ndos".splitlines()	['uno', 'dos']
rsplit()	Corta desde la derecha.	"a-b-c".rsplit("-", 1)	['a-b', 'c']
join()	Une con un separador.	"".join(["a", "b", "c"])	'a-b-c'
join() con espacio	Arma una frase.	" ".join(["Hola", "GM"])	'Hola GM'
join() sin separador	Une pegado.	"".join(["a", "b", "c"])	'abc'
A números	split + int para convertir.	list(map(int, "1 2 3".split()))	[1, 2, 3]
Quedarse con una parte	Indexa el resultado.	"ana,9".split(",")[0]	'ana'
join() con números	Convierte antes con str().	",".join(str(n) for n in [1, 2, 3])	'1,2,3'

Desempaquetado

Desempaquetar es repartir los elementos de una lista, tupla o texto en varias variables en una sola línea. Con * se recoge «el resto» en una lista. La cantidad de variables debe coincidir (o usar *).

RECUERDA Es como repartir cartas: una a cada jugador. Con *resto, el último jugador se lleva todas las que sobran.

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
a, b, c = lista	Una variable por elemento.	a, b, c = [1, 2, 3] print(a, b, c)	1 2 3
*resto	Recoge el sobrante en una lista.	a, *resto = [1, 2, 3, 4] print(a, resto)	1 [2, 3, 4]
*inicio	El * puede ir al principio.	*inicio, ultimo = [1, 2, 3, 4] print(inicio, ultimo)	[1, 2, 3] 4
En el medio	Primero, medio y último.	p, *medio, u = range(5) print(p, medio, u)	0 [1, 2, 3] 4
_	Ignora un valor que no necesitas.	a, _, c = [1, 2, 3] print(a, c)	1 3
Texto	También funciona con strings.	a, b = "xy" print(a, b)	x y
Intercambio	Cambia dos variables de lugar.	a, b = 1, 2 a, b = b, a print(a, b)	2 1

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
En un for	Desempaqueta cada par.	<pre>for n, nota in [("Ana", 9), ("Luis", 8)]: print(n, nota)</pre>	Ana 9 Luis 8
<code>print(*lista)</code>	Pasa la lista como argumentos sueltos.	<code>print(*[1, 2, 3])</code>	1 2 3
Faltan valores	Variables de más: ValueError.	<code>a, b, c = [1, 2]</code>	ValueError: not enough values to unpack (expected 3, got 2)

TEMA 44

Funciones generales

Son funciones incluidas en Python (built-in) que sirven con casi cualquier tipo de dato: len, sum, min, max, sorted, range, enumerate, zip, any, all, abs, round, etc. No necesitan import.

RECUERDA Las built-in son tu caja de herramientas gratis: ya vienen instaladas con Python.

DATOS DE EJEMPLO
`nums = [4, 9, 2, 7]`

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>len()</code>	Cantidad de elementos.	<code>len(nums)</code>	4
<code>sum()</code>	Suma de todos.	<code>sum(nums)</code>	22
<code>min()</code>	El menor.	<code>min(nums)</code>	2
<code>max()</code>	El mayor.	<code>max(nums)</code>	9
<code>sorted()</code>	Lista ordenada nueva.	<code>sorted(nums)</code>	[2, 4, 7, 9]
<code>sorted(reverse)</code>	Ordenada de mayor a menor.	<code>sorted(nums, reverse=True)</code>	[9, 7, 4, 2]
<code>reversed()</code>	Recorre al revés.	<code>list(reversed(nums))</code>	[7, 2, 9, 4]
Promedio	<code>sum()</code> entre <code>len()</code> .	<code>sum(nums) / len(nums)</code>	5.5
<code>abs()</code>	Valor absoluto.	<code>abs(-5)</code>	5
<code>pow()</code>	Potencia.	<code>pow(2, 3)</code>	8
<code>range()</code>	Secuencia de enteros.	<code>list(range(1, 6))</code>	[1, 2, 3, 4, 5]
<code>enumerate()</code>	Posición y elemento.	<code>list(enumerate(["a", "b"]))</code>	[(0, 'a'), (1, 'b')]
<code>zip()</code>	Junta dos listas en pares.	<code>list(zip([1, 2], ["a", "b"]))</code>	[(1, 'a'), (2, 'b')]
<code>any()</code>	¿Alguno cumple?	<code>any(n > 8 for n in nums)</code>	True
<code>all()</code>	¿Todos cumplen?	<code>all(n > 1 for n in nums)</code>	True
<code>map()</code>	Aplica una función a todos.	<code>list(map(str, nums))</code>	['4', '9', '2', '7']
<code>min() en texto</code>	También sirve con letras.	<code>min("hola")</code>	'a'

Módulo random

El módulo random genera números y elecciones al azar (pseudoaleatorios). Se importa con `import random`. Con `random.seed(n)` el resultado se vuelve repetible: misma semilla, mismos números.

RECUERDA `random` = tirar dados. `seed()` = dejar los dados «cargados» para que salga siempre lo mismo (útil para probar).

DATOS DE EJEMPLO

```
import random
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>random.random()</code>	Decimal entre 0 y 1.	<code>random.random()</code>	Ej: 0.8444218515250481
<code>random.randint(a, b)</code>	Entero entre a y b (incluye ambos).	<code>random.randint(1, 6)</code>	Ej: 4
<code>random.randrange()</code>	Como <code>range()</code> : inicio, fin (sin incluir), paso.	<code>random.randrange(0, 10, 2)</code>	Ej: 6 (par de 0 a 8)
<code>random.uniform(a, b)</code>	Decimal entre a y b.	<code>random.uniform(1, 5)</code>	Ej: 3.27
<code>random.choice()</code>	Elige 1 elemento.	<code>random.choice(["piedra", "papel", "tijera"])</code>	Ej: 'papel'
<code>random.choices(k=)</code>	Elige k, pueden repetirse.	<code>random.choices([1, 2, 3], k=4)</code>	Ej: [3, 1, 3, 2]
<code>random.sample()</code>	Elige k sin repetir.	<code>random.sample(range(1, 50), 3)</code>	Ej: [12, 40, 7]
<code>random.shuffle()</code>	Desordena la lista en sitio.	<pre>l = [1, 2, 3, 4] random.shuffle(l) print(l)</pre>	Ej: [3, 1, 4, 2]
<code>random.seed()</code>	Fija la semilla: resultado repetible.	<code>random.seed(1)</code> <code>print(random.randint(1, 10))</code>	3
Misma semilla	Dos tiradas iguales con la misma semilla.	<pre>random.seed(5) a = random.randint(1, 100) random.seed(5) b = random.randint(1, 100) print(a == b)</pre>	True
<code>random.gauss()</code>	Número con distribución normal.	<code>random.gauss(0, 1)</code>	Ej: -0.37
Dos dados	Suma de dos dados.	<code>random.randint(1, 6) + random.randint(1, 6)</code>	Ej: 7 (entre 2 y 12)
Moneda	Cara o sello.	<code>random.choice(["cara", "sello"])</code>	Ej: 'sello'

Las filas con «Ej:» cambian cada vez que corres el código. Las que usan `seed()` dan siempre el mismo resultado.

TEMA 46

Listas anidadas (matrices)

Una lista puede contener otras listas. Así se representan tablas o matrices: `m[fila][columna]`. Para recorrerla completa se usan dos bucles for, uno dentro de otro.

RECUERDA Matriz = edificio: primero eliges el piso (fila) y luego el departamento (columna): `m[piso][depto]`.

DATOS DE EJEMPLO

```
m = [[1, 2, 3], [4, 5, 6]]
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>m[0]</code>	Una fila completa.	<code>m[0]</code>	[1, 2, 3]
<code>m[1][2]</code>	Fila 1, columna 2.	<code>m[1][2]</code>	6

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>len(m)</code>	Cantidad de filas.	<code>len(m)</code>	2
<code>len(m[0])</code>	Cantidad de columnas.	<code>len(m[0])</code>	3
Recorrer filas	Un for simple.	<pre>for fila in m: print(fila)</pre>	[1, 2, 3] [4, 5, 6]
Dos for	Recorre cada valor.	<pre>for f in m: for x in f: print(x, end=" ")</pre>	1 2 3 4 5 6
<code>sum(m[0])</code>	Suma de una fila.	<code>sum(m[0])</code>	6
Una columna	Toma el elemento 0 de cada fila.	<code>[f[0] for f in m]</code>	[1, 4]
Total	Suma de toda la matriz.	<code>sum(sum(f) for f in m)</code>	21
Modificar	Cambia un valor.	<pre>m[0][1] = 9 print(m)</pre>	[[1, 9, 3], [4, 5, 6]]
Crear matriz de ceros	2 filas por 3 columnas.	<code>[[0] * 3 for _ in range(2)]</code>	[[0, 0, 0], [0, 0, 0]]
Trampa con *	Las filas son la misma lista.	<pre>t = [[0] * 3] * 2 t[0][0] = 1 print(t)</pre>	[[1, 0, 0], [1, 0, 0]]
Transponer	Filas ↔ columnas.	<code>list(zip(*m))</code>	[(1, 4), (2, 5), (3, 6)]

TEMA 47

Comprensión de listas

Es una forma corta de crear una lista a partir de otra: `[expresión for x in secuencia if condición]`. Reemplaza un for + append en una sola línea.

RECUERDA Léelo en español: «dame x*2 por cada x en el rango». Es un for comprimido.

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>[x * 2 for ...]</code>	Transforma cada elemento.	<code>[x * 2 for x in range(5)]</code>	[0, 2, 4, 6, 8]
Con if	Filtra solo los que cumplen.	<code>[x for x in range(10) if x % 2 == 0]</code>	[0, 2, 4, 6, 8]
Sobre texto	Una letra por elemento.	<code>[c.upper() for c in "abc"]</code>	['A', 'B', 'C']
Longitudes	Largo de cada palabra.	<code>[len(p) for p in ["hola", "GM"]]</code>	[4, 2]
if / else	Elige un valor según cada caso.	<code>["par" if x % 2 == 0 else "impar" for x in range(3)]</code>	['par', 'impar', 'par']
Convertir tipos	Texto a enteros.	<code>[int(x) for x in "1 2 3".split()]</code>	[1, 2, 3]
Cuadrados	Potencia de cada número.	<code>[n ** 2 for n in [1, 2, 3, 4]]</code>	[1, 4, 9, 16]
Diccionario	Mismo truco con {}.	<code>{x: x ** 2 for x in range(3)}</code>	{0: 0, 1: 1, 2: 4}
Conjunto	Sin repetidos.	<code>{x % 3 for x in range(10)}</code>	{0, 1, 2}
Matriz	Lista de listas.	<code>[[0] * 3 for _ in range(2)]</code>	[[0, 0, 0], [0, 0, 0]]

Control de flujo y funciones

Tomar decisiones, repetir tareas y organizar el código en funciones.

TEMA 48

Sentencias condicionales

Permiten que el programa tome decisiones. `if` ejecuta un bloque si la condición es `True`; `elif` prueba otra condición; `else` se ejecuta cuando ninguna se cumplió. El bloque se marca con dos puntos y la indentación.

RECUERDA `if` = «si», `elif` = «si no, pero si...», `else` = «si no». Solo entra UNA rama: la primera que se cumpla.

SENTENCIA	DESCRIPCIÓN	EJEMPLO	RESULTADO
if	Ejecuta si la condición es <code>True</code> .	<pre>edad = 20 if edad >= 18: print("mayor de edad")</pre>	mayor de edad
if / else	Dos caminos posibles.	<pre>n = 5 if n % 2 == 0: print("par") else: print("impar")</pre>	impar
elif	Varias condiciones en cadena.	<pre>nota = 8 if nota >= 9: print("A") elif nota >= 7: print("B") else: print("C")</pre>	B
Primera que cumple	Solo corre la primera rama verdadera.	<pre>n = 15 if n > 5: print("mayor que 5") elif n > 10: print("mayor que 10")</pre>	mayor que 5
and / or	Condiciones combinadas.	<pre>x = 15 if x > 10 and x < 20: print("entre 10 y 20")</pre>	entre 10 y 20
Anidado	Un <code>if</code> dentro de otro.	<pre>n = 4 if n > 0: if n % 2 == 0: print("positivo y par")</pre>	positivo y par
Operador ternario	<code>if / else</code> en una línea.	<pre>print("sí" if 5 > 3 else "no")</pre>	sí
in	Condición con pertenencia.	<pre>if "a" in "casa": print("tiene a")</pre>	tiene a
Lista vacía	Una lista vacía cuenta como <code>False</code> .	<pre>lista = [] if not lista: print("vacía")</pre>	vacía
Comparación encadenada	Rango en una sola condición.	<pre>x = 5 if 1 < x < 10: print("en rango")</pre>	en rango
Año bisiesto	Condición lógica completa.	<pre>a = 2024 print(a % 4 == 0 and (a % 100 != 0 or a % 400 == 0))</pre>	True
pass	Bloque vacío: no hace nada.	<pre>if True: pass print("sigue")</pre>	sigue

SENTENCIA	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>= en vez de ==</code>	Error clásico en el if.	<pre>x = 5 if x = 5: print("cinco")</pre>	SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?

TEMA 49

Bucle for

for recorre una secuencia (rango, lista, texto, diccionario...) y repite el bloque una vez por elemento. range(inicio, fin, paso) genera números: el fin no se incluye. break sale del bucle, continue salta a la siguiente vuelta y else se ejecuta si no hubo break.

RECUERDA for = «para cada». Léelo así: «para cada letra en este texto, haz esto».

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
range(n)	De 0 hasta n - 1.	<pre>for i in range(3): print(i)</pre>	0 1 2
range(a, b)	De a hasta b - 1.	<pre>for i in range(1, 4): print(i)</pre>	1 2 3
range(a, b, paso)	Con salto.	<pre>for i in range(0, 10, 3): print(i)</pre>	0 3 6 9
Cuenta regresiva	Paso negativo.	<pre>for i in range(3, 0, -1): print(i)</pre>	3 2 1
Sobre una lista	Elemento por elemento.	<pre>for f in ["pera", "uva"]: print(f)</pre>	pera uva
Sobre un texto	Letra por letra.	<pre>for c in "GM": print(c)</pre>	G M
Por índice	Usa range(len(...)).	<pre>f = ["pera", "uva"] for i in range(len(f)): print(i, f[i])</pre>	0 pera 1 uva
enumerate()	Índice y valor; start= cambia el inicio.	<pre>for i, f in enumerate(["pera", "uva"], start=1): print(i, f)</pre>	1 pera 2 uva
zip()	Recorre dos listas a la vez.	<pre>for n, nt in zip(["Ana", "Luis"], [9, 8]): print(n, nt)</pre>	Ana 9 Luis 8
Acumulador	Suma del 1 al 5.	<pre>total = 0 for n in range(1, 6): total += n print(total)</pre>	15
Contador con if	Cuenta los pares.	<pre>c = 0 for n in range(10): if n % 2 == 0: c += 1 print(c)</pre>	5
break	Salte del bucle.	<pre>for n in range(10): if n == 3: break print(n)</pre>	0 1 2
continue	Salta esta vuelta.	<pre>for n in range(5): if n == 2: continue print(n)</pre>	0 1 3 4

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
for ... else	El else corre si no hubo break.	<pre>for n in range(3): pass else: print("terminó sin break")</pre>	terminó sin break
Anidados	Un for dentro de otro.	<pre>for i in range(1, 3): for j in range(1, 3): print(i, j)</pre>	1 1 1 2 2 1 2 2
Tabla de multiplicar	Con f-string.	<pre>for i in range(1, 4): print(f"2 x {i} = {2 * i}")</pre>	2 x 1 = 2 2 x 2 = 4 2 x 3 = 6
end=" "	Todo en una línea.	<pre>for i in range(5): print(i, end=" ")</pre>	0 1 2 3 4
Triángulo de *	Patrón con texto repetido.	<pre>for i in range(1, 4): print("*" * i)</pre>	* ** ***
Diccionario	Claves y valores con items().	<pre>for k, v in {"a": 1, "b": 2}.items(): print(k, v)</pre>	a 1 b 2
_	Repetir sin usar la variable.	<pre>for _ in range(2): print("hola")</pre>	hola hola
range es perezoso	No guarda todos los números.	<pre>print(range(3))</pre>	range(0, 3)

TEMA 50

Bucle while

while repite el bloque mientras la condición sea True. Ideal cuando no sabes cuántas veces se repetirá. Siempre debe haber algo dentro que haga que la condición algún día sea False; si no, se vuelve infinito.

RECUERDA while = «mientras». Si te olvidas de cambiar la variable de la condición, el bucle no termina jamás.

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
while básico	Cuenta de 0 a 2.	<pre>i = 0 while i < 3: print(i) i += 1</pre>	0 1 2
Cuenta regresiva	Baja hasta 1.	<pre>n = 3 while n > 0: print(n) n -= 1 print("¡Despegue!")</pre>	3 2 1 ¡Despegue!
Acumulador	Suma del 1 al 5.	<pre>total, n = 0, 1 while n <= 5: total += n n += 1 print(total)</pre>	15
while True + break	Bucle que tú decides cuándo cortar.	<pre>n = 0 while True: n += 1 if n == 3: break print(n)</pre>	3
continue	Salta a la siguiente vuelta.	<pre>i = 0 while i < 5: i += 1 if i == 3: continue print(i)</pre>	1 2 4 5

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
while ... else	El else corre si no hubo break.	<pre>i = 0 while i < 2: i += 1 else: print("fin normal")</pre>	fin normal
Suma de dígitos	Con % y //.	<pre>n = 1234 s = 0 while n > 0: s += n % 10 n //= 10 print(s)</pre>	10
Factorial	Multiplica hacia abajo.	<pre>n, f = 5, 1 while n > 1: f *= n n -= 1 print(f)</pre>	120
Potencias de 2	Hasta pasar de 100.	<pre>p = 1 while p < 100: p *= 2 print(p)</pre>	128
Validar entrada	Repita hasta que sea válida.	<pre>n = int(input("Positivo: ")) while n <= 0: n = int(input("Otra vez: ")) print("ok", n)</pre>	Positivo: -5 Otra vez: 7 ok 7
Centinela	Termina cuando el usuario escribe 0.	<pre>s = 0 x = int(input("Número: ")) while x != 0: s += x x = int(input("Número: ")) print(s)</pre>	Número: 4 Número: 6 Número: 0 10
Vaciar una lista	Mientras tenga elementos.	<pre>cola = [1, 2, 3] while cola: print(cola.pop(0))</pre>	1 2 3
Bucle infinito	Falta cambiar la condición.	<pre>while True: print("hola")</pre>	Imprime «hola» sin parar (Ctrl + C para detener)

TEMA 51

Creación de funciones

Una función es un bloque de código con nombre que se puede reutilizar. Se define con `def`, puede recibir parámetros y devolver un valor con `return`. Lo que se crea dentro de la función (variables locales) no existe fuera de ella.

RECUERDA Función = receta con nombre. La defines una vez (`def`) y la «cocinas» las veces que quieras llamándola.

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
def	Define una función sin parámetros.	<pre>def saludar(): print("Hola") saludar()</pre>	Hola
Parámetro	Recibe un dato al llamarla.	<pre>def saludar(nombre): print("Hola", nombre) saludar("Ana")</pre>	Hola Ana
return	Devuelve un resultado.	<pre>def doble(n): return n * 2 print(doble(5))</pre>	10
Varios parámetros	Se separan con comas.	<pre>def suma(a, b): return a + b print(suma(3, 4))</pre>	7

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Valor por defecto	El parámetro es opcional.	<pre>def saludo(n="amigo"): return "Hola " + n print(saludo())</pre>	Hola amigo
Argumentos con nombre	El orden deja de importar.	<pre>def resta(a, b): return a - b print(resta(b=2, a=10))</pre>	8
Devolver varios	Devuelve una tupla.	<pre>def mm(l): return min(l), max(l) print(mm([4, 9, 2]))</pre>	(2, 9)
Sin return	Devuelve None.	<pre>def f(): x = 1 print(f())</pre>	None
print vs return	return te deja usar el valor.	<pre>def b(): return 5 print(b() + 1)</pre>	6
Variable local	No existe fuera de la función.	<pre>def f(): z = 1 f() print(z)</pre>	NameError: name 'z' is not defined
global	Modifica una variable de afuera.	<pre>c = 0 def sumar(): global c c += 1 sumar() print(c)</pre>	1
Docstring	Texto que explica la función.	<pre>def area(r): """Área de un círculo.""" return 3.14 * r ** 2 print(area.__doc__)</pre>	Área de un círculo.
*args	Cantidad variable de argumentos.	<pre>def total(*nums): return sum(nums) print(total(1, 2, 3))</pre>	6
kwargs	Argumentos con nombre variables.	<pre>def datos(kw): return kw print(datos(a=1, b=2))</pre>	{'a': 1, 'b': 2}
lambda	Función corta de una línea.	<pre>doble = lambda n: n * 2 print(doble(4))</pre>	8
Recursión	Una función que se llama a sí misma.	<pre>def fact(n): return 1 if n <= 1 else n * fact(n - 1) print(fact(5))</pre>	120
Llamar antes de definir	Primero def, después la llamada.	<pre>saludar() def saludar(): print("hola")</pre>	NameError: name 'saludar' is not defined

TEMA 52

Excepciones (try / except)

Una excepción es un error que ocurre mientras el programa corre. Con try / except puedes atraparlo y seguir en lugar de que el programa se caiga. else corre si no hubo error y finally corre siempre.

RECUERDA try = «intenta esto». except = «si falla, haz esto otro». finally = «pase lo que pase, haz esto».

BLOQUE	DESCRIPCIÓN	EJEMPLO	RESULTADO
try / except	Atrapa un error concreto.	<pre>try: print(10 / 0) except ZeroDivisionError: print("No se divide entre 0")</pre>	No se divide entre 0

BLOQUE	DESCRIPCIÓN	EJEMPLO	RESULTADO
ValueError	Texto que no es número.	<pre>try: n = int("abc") except ValueError: print("No es un número")</pre>	No es un número
as e	Muestra el mensaje del error.	<pre>try: int("x") except ValueError as e: print(e)</pre>	invalid literal for int() with base 10: 'x'
else	Corre si NO hubo error.	<pre>try: n = int("5") except ValueError: print("mal") else: print("bien", n)</pre>	bien 5
finally	Corre siempre.	<pre>try: print(1) finally: print("siempre")</pre>	1 siempre
Varios errores	Una tupla de excepciones.	<pre>try: [1][3] except (IndexError, KeyError): print("no existe")</pre>	no existe
raise	Lanza tu propio error.	<pre>try: raise ValueError("dato inválido") except ValueError as e: print(e)</pre>	dato inválido
Validar entrada	Repite hasta que escriban un número.	<pre>while True: try: n = int(input("Número: ")) break except ValueError: print("De nuevo") print(n)</pre>	Número: a De nuevo Número: 8 8
except Exception	Atrapa cualquier error.	<pre>try: {}["a"] except Exception as e: print(type(e).__name__)</pre>	KeyError

Archivos y estructuras de datos

Guardar información en el disco y organizar datos con tuplas, diccionarios y conjuntos.

TEMA 53

Archivos

`open(nombre, modo)` abre un archivo. Modos: "r" leer (por defecto), "w" escribir (borra lo anterior), "a" agregar al final, "x" crear (da error si ya existe). Con `with open(...)` as `f` el archivo se cierra solo al terminar el bloque.

RECUERDA Siempre usa `with`: es como una puerta que se cierra sola. Si usas `open()` a mano, no olvides `f.close()`.

DATOS DE EJEMPLO
 Archivo `notas.txt`:
 Ana,9
 Luis,8
 Mia,10

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>open() + read() + close()</code>	Forma manual: abrir, leer y cerrar.	<pre>f = open("notas.txt") texto = f.read() f.close() print(texto)</pre>	Ana,9 Luis,8 Mia,10
<code>with open()</code>	Se cierra solo al terminar.	<pre>with open("notas.txt") as f: print(f.read())</pre>	Ana,9 Luis,8 Mia,10
<code>f.closed</code>	¿Quedó cerrado?	<pre>with open("notas.txt") as f: pass print(f.closed)</pre>	True
<code>read(n)</code>	Lee solo n caracteres.	<pre>with open("notas.txt") as f: print(f.read(3))</pre>	Ana
<code>readline()</code>	Lee una línea.	<pre>with open("notas.txt") as f: print(f.readline().strip())</pre>	Ana,9
<code>readlines()</code>	Todas las líneas en una lista.	<pre>with open("notas.txt") as f: print(f.readlines())</pre>	['Ana,9\n', 'Luis,8\n', 'Mia,10\n']
<code>for linea in f</code>	Recorre línea por línea.	<pre>with open("notas.txt") as f: for linea in f: print(linea.strip())</pre>	Ana,9 Luis,8 Mia,10
Separar datos	<code>split()</code> para sacar las columnas.	<pre>with open("notas.txt") as f: for linea in f: n, nota = linea.strip().split(",") print(n, int(nota) + 1)</pre>	Ana 10 Luis 9 Mia 11
"w" escribir	Crea el archivo (o borra lo que tenía).	<pre>with open("nuevo.txt", "w") as f: f.write("hola\n") with open("nuevo.txt") as f: print(f.read())</pre>	hola
"a" agregar	Añade al final sin borrar.	<pre>with open("notas.txt", "a") as f: f.write("Leo,7\n") with open("notas.txt") as f: print(len(f.readlines()))</pre>	4
<code>writelines()</code>	Escribe una lista de líneas.	<pre>with open("l.txt", "w") as f: f.writelines(["a\n", "b\n"]) with open("l.txt") as f: print(f.read())</pre>	a b

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
encoding	Para tildes y ñ usa utf-8.	<pre>with open("t.txt", "w", encoding="utf-8") as f: f.write("año") with open("t.txt", encoding="utf-8") as f: print(f.read())</pre>	año
"x" crear	Da error si el archivo ya existe.	<pre>open("notas.txt", "x")</pre>	FileExistsError: [Errno 17] File exists: 'notas.txt'
FileNotFoundError	El archivo no existe.	<pre>open("noexiste.txt")</pre>	FileNotFoundError: [Errno 2] No such file or directory: 'noexiste.txt'
os.path.exists()	¿Existe el archivo?	<pre>import os print(os.path.exists("notas.txt"))</pre>	True
os.remove()	Borra un archivo.	<pre>import os os.remove("notas.txt") print(os.path.exists("notas.txt"))</pre>	False

TEMA 54

Tipo de dato: tupla

Una tupla es como una lista pero inmutable: una vez creada no se puede cambiar. Se escribe con paréntesis (). Se usa para datos que no deben modificarse (coordenadas, fechas) y para devolver varios valores. Una tupla de un solo elemento necesita coma: (5,).

RECUERDA Tupla = lista «sellada». Puedes mirar todo lo que hay adentro, pero no cambiar nada.

DATOS DE EJEMPLO

```
t = (4, 8, 15, 8)
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
()	Crear una tupla.	<pre>(1, 2, 3)</pre>	(1, 2, 3)
Sin paréntesis	La coma es lo que la crea.	<pre>t = 1, 2, 3 print(type(t))</pre>	<class 'tuple'>
Un elemento	Necesita coma final.	<pre>print(type((5,)), type((5)))</pre>	<class 'tuple'> <class 'int'>
t[0]	Primer elemento.	<pre>t[0]</pre>	4
t[-1]	Último elemento.	<pre>t[-1]</pre>	8
t[1:3]	Slicing: devuelve otra tupla.	<pre>t[1:3]</pre>	(8, 15)
len()	Cantidad de elementos.	<pre>len(t)</pre>	4
count()	Cuántas veces aparece.	<pre>t.count(8)</pre>	2
index()	Posición del elemento.	<pre>t.index(15)</pre>	2
+	Une tuplas.	<pre>t + (1,)</pre>	(4, 8, 15, 8, 1)
*	Repite la tupla.	<pre>(1, 2) * 2</pre>	(1, 2, 1, 2)
in	¿Está el elemento?	<pre>8 in t</pre>	True
min / max / sum	Funciones generales.	<pre>print(min(t), max(t), sum(t))</pre>	4 15 35
sorted()	Devuelve una lista ordenada.	<pre>sorted(t)</pre>	[4, 8, 8, 15]
Inmutable	No se puede cambiar un elemento.	<pre>t[0] = 9</pre>	TypeError: 'tuple' object does not support item assignment

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
del	Tampoco se puede borrar.	<code>del t[0]</code>	<code>TypeError: 'tuple' object doesn't support item deletion</code>
Desempaquetar	Reparte en variables.	<code>a, b, c, d = t print(a, d)</code>	4 8
tuple()	Convierte una lista en tupla.	<code>tuple([1, 2, 3])</code>	(1, 2, 3)
Anidadas	Tupla dentro de tupla.	<code>((1, 2), (3, 4))[1][0]</code>	3
Como clave	Sí sirven de clave de diccionario.	<code>{(1, 2): "punto"}[(1, 2)]</code>	'punto'
Contiene una lista	La lista de adentro sí cambia.	<code>x = ([1], 2) x[0].append(9) print(x)</code>	([1, 9], 2)

TEMA 55

Tipo de dato: diccionario

Un diccionario guarda pares clave: valor entre llaves {}. Se accede por la clave, no por la posición. Las claves son únicas e inmutables (texto, número o tupla) y los valores pueden ser de cualquier tipo.

RECUERDA Diccionario = agenda de contactos: buscas por nombre (la clave) y obtienes el teléfono (el valor).

DATOS DE EJEMPLO

```
d = {"nombre": "Ana", "edad": 19}
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
d[clave]	Obtiene un valor.	<code>d["nombre"]</code>	'Ana'
get()	Valor o None si no existe.	<code>print(d.get("curso"))</code>	None
get(clave, defecto)	Valor por defecto si no existe.	<code>d.get("curso", "sin curso")</code>	'sin curso'
Agregar	Una clave nueva.	<code>d["curso"] = "FP" print(d)</code>	{'nombre': 'Ana', 'edad': 19, 'curso': 'FP'}
Modificar	Cambia un valor existente.	<code>d["edad"] = 20 print(d["edad"])</code>	20
del	Borra un par.	<code>del d["edad"] print(d)</code>	{'nombre': 'Ana'}
in	Busca entre las claves.	<code>"edad" in d</code>	True
in con valor	Los valores no cuentan.	<code>"Ana" in d</code>	False
len()	Cantidad de pares.	<code>len(d)</code>	2
Recorrer	for pasa por las claves.	<code>for k in d: print(k, d[k])</code>	nombre Ana edad 19
dict()	Otra forma de crearlo.	<code>dict(a=1, b=2)</code>	{'a': 1, 'b': 2}
Anidado	Diccionario dentro de otro.	<code>est = {"ana": {"nota": 9}} print(est["ana"]["nota"])</code>	9
Contar elementos	Patrón clásico con get().	<code>conteo = {} for c in "casa": conteo[c] = conteo.get(c, 0) + 1 print(conteo)</code>	{'c': 1, 'a': 2, 's': 1}
KeyError	La clave no existe.	<code>d["curso"]</code>	<code>KeyError: 'curso'</code>
Clave inválida	Una lista no puede ser clave.	<code>{[1]: 2}</code>	<code>TypeError: unhashable type: 'list'</code>

Funciones de diccionarios

Los métodos de diccionario sirven para consultar y modificar los pares: `keys()` las claves, `values()` los valores, `items()` los pares, además de `get`, `update`, `pop`, `setdefault`, etc.

RECUERDA `keys()` = solo los nombres, `values()` = solo los teléfonos, `items()` = nombre y teléfono juntos.

DATOS DE EJEMPLO

```
d = {"a": 1, "b": 2, "c": 3}
```

MÉTODO	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>keys()</code>	Todas las claves.	<code>list(d.keys())</code>	<code>['a', 'b', 'c']</code>
<code>values()</code>	Todos los valores.	<code>list(d.values())</code>	<code>[1, 2, 3]</code>
<code>items()</code>	Pares (clave, valor).	<code>list(d.items())</code>	<code>[('a', 1), ('b', 2), ('c', 3)]</code>
<code>for k, v in items()</code>	Recorre claves y valores.	<pre>for k, v in d.items(): print(k, v)</pre>	<code>a 1</code> <code>b 2</code> <code>c 3</code>
<code>get()</code>	Valor sin riesgo de error.	<code>d.get("z", 0)</code>	<code>0</code>
<code>setdefault()</code>	Agrega si no existe.	<pre>d.setdefault("z", 0) print(d)</pre>	<code>{'a': 1, 'b': 2, 'c': 3, 'z': 0}</code>
<code>update()</code>	Agrega o actualiza varios.	<pre>d.update({"b": 20, "d": 4}) print(d)</pre>	<code>{'a': 1, 'b': 20, 'c': 3, 'd': 4}</code>
<code>pop()</code>	Saca y devuelve un valor.	<pre>x = d.pop("a") print(x, d)</pre>	<code>1 {'b': 2, 'c': 3}</code>
<code>popitem()</code>	Saca el último par.	<code>print(d.popitem())</code>	<code>('c', 3)</code>
<code>clear()</code>	Vacía el diccionario.	<pre>d.clear() print(d)</pre>	<code>{}</code>
<code>copy()</code>	Copia independiente.	<pre>c = d.copy() c["a"] = 99 print(d["a"])</pre>	<code>1</code>
<code>fromkeys()</code>	Crea con un valor inicial.	<code>dict.fromkeys(["x", "y"], 0)</code>	<code>{'x': 0, 'y': 0}</code>
<code>sum(values())</code>	Suma de todos los valores.	<code>sum(d.values())</code>	<code>6</code>
<code>max(d, key=d.get)</code>	Clave con el mayor valor.	<code>max(d, key=d.get)</code>	<code>'c'</code>
Ordenar por valor	De mayor a menor.	<code>sorted(d.items(), key=lambda p: p[1], reverse=True)</code>	<code>[('c', 3), ('b', 2), ('a', 1)]</code>
Invertir	Claves ↔ valores.	<code>{v: k for k, v in d.items()}</code>	<code>{1: 'a', 2: 'b', 3: 'c'}</code>
<code>d otro</code>	Une diccionarios (Python 3.9+).	<code>d {"z": 9}</code>	<code>{'a': 1, 'b': 2, 'c': 3, 'z': 9}</code>

Conjuntos (set)

Un conjunto guarda elementos únicos y sin orden, entre llaves {}. Sirve para eliminar repetidos y para operaciones de matemáticas de conjuntos: unión, intersección y diferencia. No tiene índices.

RECUERDA Conjunto = lista de invitados sin repetir. Si anotas dos veces a Ana, solo queda una.

DATOS DE EJEMPLO

```
a = {1, 2, 3}
b = {3, 4}
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>{ }</code>	Crea un conjunto. Quita los repetidos.	<code>{1, 2, 2, 3}</code>	<code>{1, 2, 3}</code>
<code>set()</code>	Conjunto vacío (ojo: {} es un dict).	<code>set()</code>	<code>set()</code>
<code>add()</code>	Agrega un elemento.	<code>a.add(9)</code> <code>print(a)</code>	<code>{1, 2, 3, 9}</code>
<code>remove()</code>	Quita (error si no está).	<code>a.remove(1)</code> <code>print(a)</code>	<code>{2, 3}</code>
<code>discard()</code>	Quita (sin error si no está).	<code>a.discard(99)</code> <code>print(a)</code>	<code>{1, 2, 3}</code>
<code>a b</code>	Unión: todo junto.	<code>a b</code>	<code>{1, 2, 3, 4}</code>
<code>a & b</code>	Intersección: lo común.	<code>a & b</code>	<code>{3}</code>
<code>a - b</code>	Diferencia: está en a pero no en b.	<code>a - b</code>	<code>{1, 2}</code>
<code>a ^ b</code>	Está en uno solo de los dos.	<code>a ^ b</code>	<code>{1, 2, 4}</code>
<code>in</code>	Pertenencia (muy rápido).	<code>3 in a</code>	<code>True</code>
<code>len()</code>	Cantidad de elementos.	<code>len(a)</code>	<code>3</code>
<code>issubset()</code>	¿Está contenido en otro?	<code>a.issubset({1, 2, 3, 4})</code>	<code>True</code>
Quitar repetidos	De lista a conjunto y de vuelta.	<code>sorted(set([3, 1, 3, 2]))</code>	<code>[1, 2, 3]</code>
Letras distintas	Cuántas letras diferentes.	<code>len(set("banana"))</code>	<code>3</code>
Sin índices	No se puede hacer <code>a[0]</code> .	<code>a[0]</code>	<code>TypeError: 'set' object is not subscriptable</code>

Ciencia de datos: NumPy, Pandas y Matplotlib

Las tres librerías base para analizar números, tablas y gráficos.

TEMA 58

NumPy: arreglos

NumPy es la librería de cálculo numérico de Python. Su objeto principal es el arreglo (ndarray): una colección de números del mismo tipo, mucho más rápida que una lista. Se instala con `pip install numpy` y se importa como `np`.

RECUERDA Lista = cajón mezclado y lento. Arreglo NumPy = bandeja de huevos: todos iguales, ordenados y rapidísimos.

DATOS DE EJEMPLO

```
import numpy as np
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>np.array()</code>	Crea un arreglo desde una lista.	<code>np.array([1, 2, 3])</code>	<code>[1 2 3]</code>
<code>np.array()</code> 2D	Lista de listas = matriz.	<code>np.array([[1, 2], [3, 4]])</code>	<code>[[1 2] [3 4]]</code>
<code>np.zeros()</code>	Arreglo lleno de ceros.	<code>np.zeros(3)</code>	<code>[0. 0. 0.]</code>
<code>np.ones()</code>	Arreglo lleno de unos (2 filas, 3 columnas).	<code>np.ones((2, 3))</code>	<code>[[1. 1. 1.] [1. 1. 1.]]</code>
<code>np.full()</code>	Lleno de un valor.	<code>np.full(3, 7)</code>	<code>[7 7 7]</code>
<code>np.arange()</code>	Como <code>range()</code> , pero devuelve un arreglo.	<code>np.arange(0, 10, 3)</code>	<code>[0 3 6 9]</code>
<code>np.linspace()</code>	N puntos parejos entre dos valores.	<code>np.linspace(0, 1, 5)</code>	<code>[0. 0.25 0.5 0.75 1.]</code>
<code>np.eye()</code>	Matriz identidad.	<code>np.eye(2)</code>	<code>[[1. 0.] [0. 1.]]</code>
<code>reshape()</code>	Cambia la forma.	<code>np.arange(6).reshape(2, 3)</code>	<code>[[0 1 2] [3 4 5]]</code>
<code>dtype=</code>	Elige el tipo de dato.	<code>np.array([1, 2, 3], dtype=float)</code>	<code>[1. 2. 3.]</code>
Mezcla ints y floats	Todo pasa a float.	<code>np.array([1, 2.5])</code>	<code>[1. 2.5]</code>
<code>np.random.randint()</code>	Enteros al azar (con semilla).	<code>np.random.seed(0)</code> <code>np.random.randint(1, 10, 4)</code>	<code>[6 1 4 4]</code>
<code>np.zeros_like()</code>	Ceros con la forma de otro arreglo.	<code>np.zeros_like(np.array([5, 6, 7]))</code>	<code>[0 0 0]</code>

NumPy: operaciones numéricas

Con NumPy las operaciones se aplican elemento por elemento, sin escribir bucles (vectorización). También puedes operar un arreglo con un solo número (broadcasting). Con listas normales + y * hacen otra cosa: unir y repetir.

RECUERDA Un arreglo se comporta como una fila de calculadoras: cada una opera su propio número al mismo tiempo.

DATOS DE EJEMPLO

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
a + b	Suma elemento a elemento.	a + b	[11 22 33]
a * b	Multiplica elemento a elemento.	a * b	[10 40 90]
b / a	División elemento a elemento.	b / a	[10. 10. 10.]
a ** 2	Potencia a cada elemento.	a ** 2	[1 4 9]
a + 10	Un número se aplica a todos.	a + 10	[11 12 13]
a * 2	Con arreglos multiplica...	a * 2	[2 4 6]
lista * 2	...con listas repite.	[1, 2, 3] * 2	[1, 2, 3, 1, 2, 3]
b // 7, b % 7	División entera y residuo.	print(b // 7, b % 7)	[1 2 4] [3 6 2]
np.dot()	Producto punto.	np.dot(a, b)	140
A @ A	Multiplicación de matrices.	A = np.array([[1, 2], [3, 4]]) A @ A	[[7 10] [15 22]]
np.sqrt()	Raíz de cada elemento.	np.sqrt(a)	[1. 1.41421356 1.73205081]
np.exp()	Exponencial e^x.	np.exp(a)	[2.71828183 7.3890561 20.08553692]
np.log()	Logaritmo natural.	np.log(a)	[0. 0.69314718 1.09861229]
np.abs()	Valor absoluto.	np.abs(np.array([-1, 2, -3]))	[1 2 3]
np.round()	Redondea cada elemento.	np.round(np.sqrt(a), 2)	[1. 1.41 1.73]
np.maximum()	Mayor entre arreglo y valor.	np.maximum(a, 2)	[2 2 3]
Formas distintas	Los tamaños deben ser compatibles.	a + np.array([1, 2])	ValueError: operands could not be broadcast together with shapes (3,) (2,)

NumPy: estadísticas y cálculos eficientes

Los arreglos traen métodos estadísticos listos: suma, promedio, mediana, desviación, mínimo y máximo. En matrices, `axis=0` opera por columnas y `axis=1` por filas. Todo es mucho más rápido que un `for`.

RECUERDA `axis=0` → baja por las columnas (↓). `axis=1` → camina por las filas (→).

DATOS DE EJEMPLO

```
import numpy as np
v = np.array([4, 8, 15, 16, 23, 42])
m = np.array([[1, 2, 3], [4, 5, 6]])
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>v.sum()</code>	Suma total.	<code>v.sum()</code>	108
<code>v.mean()</code>	Promedio.	<code>v.mean()</code>	18.0
<code>np.median()</code>	Mediana (valor del medio).	<code>np.median(v)</code>	15.5
<code>v.std()</code>	Desviación estándar.	<code>v.std()</code>	12.315302134607444
<code>v.var()</code>	Varianza.	<code>v.var()</code>	151.66666666666666
<code>v.min()</code> / <code>v.max()</code>	Menor y mayor.	<code>print(v.min(), v.max())</code>	4 42
<code>v.argmax()</code>	Posición del mayor.	<code>v.argmax()</code>	5
<code>v.argmin()</code>	Posición del menor.	<code>v.argmin()</code>	0
<code>v.cumsum()</code>	Suma acumulada.	<code>v.cumsum()</code>	[4 12 27 43 66 108]
<code>np.diff()</code>	Diferencia entre vecinos.	<code>np.diff(v)</code>	[4 7 1 7 19]
<code>np.percentile()</code>	Percentil 50 = mediana.	<code>np.percentile(v, 50)</code>	15.5
<code>np.ptp()</code>	Rango: máximo menos mínimo.	<code>np.ptp(v)</code>	38
<code>m.sum(axis=0)</code>	Suma por columnas.	<code>m.sum(axis=0)</code>	[5 7 9]
<code>m.sum(axis=1)</code>	Suma por filas.	<code>m.sum(axis=1)</code>	[6 15]
<code>m.mean(axis=1)</code>	Promedio de cada fila.	<code>m.mean(axis=1)</code>	[2. 5.]
Sin bucles	Operar todo de una sola vez.	<code>(v * 2).sum()</code>	216

NumPy: propiedades, funciones y métodos en vectores

Las propiedades describen el arreglo (`shape`, `ndim`, `size`, `dtype`) y no llevan paréntesis. Los métodos y funciones lo transforman o combinan (`reshape`, `concatenate`, `unique`, `sort...`).

RECUERDA Propiedad = «datos de la etiqueta» (sin paréntesis). Método = «acción» (con paréntesis).

DATOS DE EJEMPLO

```
import numpy as np
v = np.array([4, 8, 15, 16, 23, 42])
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>v.shape</code>	Forma del arreglo: un vector da una sola medida.	<code>v.shape</code>	(6,)
<code>v.ndim</code>	Número de dimensiones.	<code>v.ndim</code>	1
<code>v.size</code>	Cantidad total de elementos.	<code>v.size</code>	6
<code>v.dtype</code>	Tipo de dato guardado.	<code>v.dtype</code>	int64

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>len(v)</code>	Longitud del primer eje.	<code>len(v)</code>	6
<code>v.astype()</code>	Cambia el tipo.	<code>v.astype(float)</code>	[4. 8. 15. 16. 23. 42.]
<code>v.reshape()</code>	Cambia la forma.	<code>v.reshape(2, 3)</code>	[[4 8 15] [16 23 42]]
<code>flatten()</code>	Aplana a una dimensión.	<code>v.reshape(2, 3).flatten()</code>	[4 8 15 16 23 42]
<code>v.T</code>	Transpuesta.	<code>v.reshape(2, 3).T</code>	[[4 16] [8 23] [15 42]]
<code>np.concatenate()</code>	Une arreglos.	<code>np.concatenate([v, [1, 2]])</code>	[4 8 15 16 23 42 1 2]
<code>np.append()</code>	Agrega al final.	<code>np.append(v, 99)</code>	[4 8 15 16 23 42 99]
<code>np.insert()</code>	Inserta en una posición.	<code>np.insert(v, 1, 99)</code>	[4 99 8 15 16 23 42]
<code>np.delete()</code>	Quita por posición.	<code>np.delete(v, 0)</code>	[8 15 16 23 42]
<code>np.sort()</code>	Ordena (copia).	<code>np.sort(np.array([3, 1, 2]))</code>	[1 2 3]
<code>np.unique()</code>	Valores sin repetir.	<code>np.unique([1, 2, 2, 3, 3, 3])</code>	[1 2 3]
<code>np.split()</code>	Divide en partes iguales.	<code>np.split(v, 3)</code>	[array([4, 8]), array([15, 16]), array([23, 42])]
<code>v.tolist()</code>	De arreglo a lista.	<code>v.tolist()</code>	[4, 8, 15, 16, 23, 42]
<code>v.copy()</code>	Copia independiente.	<code>np.array_equal(v, v.copy())</code>	True

TEMA 62

NumPy: indexación de posiciones

Se indexa como en las listas ([]), pero en matrices se pueden usar dos índices: `m[fila, columna]`. Los dos puntos : significan «todo». Un slice NO copia: es una vista del arreglo original.

RECUERDA `m[fila, columna]` y los dos puntos «:» significan «todos». `m[:, 1]` = «todas las filas de la columna 1».

DATOS DE EJEMPLO

```
import numpy as np
v = np.array([10, 20, 30, 40, 50])
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>v[0]</code>	Primer elemento.	<code>v[0]</code>	10
<code>v[-1]</code>	Último elemento.	<code>v[-1]</code>	50
<code>v[1:4]</code>	Slicing.	<code>v[1:4]</code>	[20 30 40]
<code>v[::2]</code>	De 2 en 2.	<code>v[::2]</code>	[10 30 50]
<code>v[::-1]</code>	Al revés.	<code>v[::-1]</code>	[50 40 30 20 10]
<code>v[[0, 2, 4]]</code>	Varias posiciones a la vez.	<code>v[[0, 2, 4]]</code>	[10 30 50]
<code>v[0] = 99</code>	Cambia un elemento.	<code>v[0] = 99</code> <code>print(v)</code>	[99 20 30 40 50]
<code>v[1:3] = 0</code>	Cambia un pedazo.	<code>v[1:3] = 0</code> <code>print(v)</code>	[10 0 0 40 50]
<code>m[1, 2]</code>	Fila 1, columna 2.	<code>m[1, 2]</code>	6
<code>m[0]</code>	Una fila completa.	<code>m[0]</code>	[1 2 3]
<code>m[:, 1]</code>	Una columna completa.	<code>m[:, 1]</code>	[2 5 8]

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>m[0:2, 1:]</code>	Un bloque de la matriz.	<code>m[0:2, 1:]</code>	<pre>[[2 3] [5 6]]</pre>
<code>m[[0, 2]]</code>	Filas elegidas.	<code>m[[0, 2]]</code>	<pre>[[1 2 3] [7 8 9]]</pre>
<code>np.diag()</code>	Diagonal principal.	<code>np.diag(m)</code>	<pre>[1 5 9]</pre>
Fuera de rango	Da <code>IndexError</code> .	<code>v[10]</code>	<code>IndexError: index 10 is out of bounds for axis 0 with size 5</code>
Un slice es una vista	Cambiarlo cambia el original.	<pre>w = v[1:3] w[0] = 0 print(v)</pre>	<pre>[10 0 30 40 50]</pre>

TEMA 63

NumPy: comparación e indexación booleana

Comparar un arreglo (`v > 10`) devuelve otro arreglo de `True/False`. Ese arreglo sirve como filtro: `v[v > 10]` deja solo los que cumplen. Para combinar condiciones se usa `&` (y), `|` (o), `~` (no) con paréntesis; `NO` and / or.

RECUERDA Es un colador: la comparación crea la malla (`True/False`) y `v[malla]` deja pasar solo los `True`.

DATOS DE EJEMPLO

```
import numpy as np
v = np.array([4, 8, 15, 16, 23, 42])
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>v > 10</code>	<code>True/False</code> por elemento.	<code>v > 10</code>	<pre>[False False True True True True]</pre>
<code>v == 8</code>	Igualdad elemento a elemento.	<code>v == 8</code>	<pre>[False True False False False False]</pre>
<code>v[v > 10]</code>	Filtra los que cumplen.	<code>v[v > 10]</code>	<pre>[15 16 23 42]</pre>
<code>&</code> (y)	Dos condiciones a la vez.	<code>v[(v > 5) & (v < 20)]</code>	<pre>[8 15 16]</pre>
<code> </code> (o)	Una u otra condición.	<code>v[(v < 5) (v > 30)]</code>	<pre>[4 42]</pre>
<code>~</code> (no)	Niega la condición.	<code>v[~(v > 10)]</code>	<pre>[4 8]</pre>
Pares	Con el residuo.	<code>v[v % 2 == 0]</code>	<pre>[4 8 16 42]</pre>
<code>np.sum(v > 10)</code>	Cuántos cumplen.	<code>np.sum(v > 10)</code>	<pre>4</pre>
<code>(v > 10).mean()</code>	Proporción que cumple.	<code>(v > 10).mean()</code>	<pre>0.6666666666666666</pre>
<code>np.any()</code>	¿Alguno cumple?	<code>np.any(v > 40)</code>	<pre>True</pre>
<code>np.all()</code>	¿Todos cumplen?	<code>np.all(v > 0)</code>	<pre>True</pre>
<code>np.where(c, a, b)</code>	Elige a o b según la condición.	<code>np.where(v > 10, 1, 0)</code>	<pre>[0 0 1 1 1 1]</pre>
<code>np.where(c)</code>	Posiciones que cumplen.	<code>np.where(v > 10)</code>	<pre>(array([2, 3, 4, 5]),)</pre>
<code>np.isin()</code>	¿Está en esta lista?	<code>np.isin(v, [8, 42])</code>	<pre>[False True False False False True]</pre>
<code>np.count_nonzero()</code>	Cuenta los <code>True</code> .	<code>np.count_nonzero(v > 10)</code>	<pre>4</pre>
<code>v[v > 10] = 0</code>	Reemplaza los que cumplen.	<pre>v[v > 10] = 0 print(v)</pre>	<pre>[4 8 0 0 0 0]</pre>
Usar and / or	No funciona con arreglos.	<code>v > 5 and v < 20</code>	<code>ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()</code>

Pandas: Series y DataFrame

Pandas trabaja con tablas. Una Series es una sola columna y un DataFrame es la tabla completa, con filas (índice) y columnas con nombre. Se importa como pd.

RECUERDA DataFrame = hoja de Excel dentro de Python. Series = una sola columna de esa hoja.

DATOS DE EJEMPLO

```
import pandas as pd
df = pd.DataFrame({
    "nombre": ["Ana", "Luis", "Mia", "Leo"],
    "nota": [9, 6, 10, 7],
    "curso": ["A", "B", "A", "B"]})
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
pd.Series()	Una columna de datos.	pd.Series([10, 20, 30])	<pre>0 10 1 20 2 30 dtype: int64</pre>
pd.DataFrame()	Tabla desde un diccionario.	df	<pre> nombre nota curso 0 Ana 9 A 1 Luis 6 B 2 Mia 10 A 3 Leo 7 B</pre>
Desde listas	Filas como listas y nombres de columnas.	pd.DataFrame([[1, 2], [3, 4]], columns=["a", "b"])	<pre> a b 0 1 2 1 3 4</pre>
df.shape	(filas, columnas).	df.shape	(4, 3)
len(df)	Cantidad de filas.	len(df)	4
df.size	Total de celdas.	df.size	12
df.columns	Nombres de columnas.	list(df.columns)	['nombre', 'nota', 'curso']
df.index	El índice de filas.	df.index	RangeIndex(start=0, stop=4, step=1)
df["nota"]	Una columna (es una Series).	df["nota"]	<pre>0 9 1 6 2 10 3 7 Name: nota, dtype: int64</pre>
df.head(n)	Primeras n filas.	df.head(2)	<pre> nombre nota curso 0 Ana 9 A 1 Luis 6 B</pre>
df.tail(n)	Últimas n filas.	df.tail(2)	<pre> nombre nota curso 2 Mia 10 A 3 Leo 7 B</pre>
df["nota"].dtype	Tipo de una columna.	df["nota"].dtype	int64
describe()	Resumen estadístico.	df["nota"].describe()	<pre>count 4.000000 mean 8.000000 std 1.825742 min 6.000000 25% 6.750000 50% 8.000000 75% 9.250000 max 10.000000 Name: nota, dtype: float64</pre>

Pandas: cargar datos (CSV y Excel)

`pd.read_csv()` lee archivos .csv y `pd.read_excel()` lee .xlsx. Ambos devuelven un DataFrame. Con `to_csv()` y `to_excel()` se guarda el resultado. Usa `index=False` para no guardar la columna del índice.

RECUERDA `read_` = leer hacia Python. `to_` = guardar desde Python.

DATOS DE EJEMPLO

```
import pandas as pd
```

Archivo notas.csv:

nombre,nota,curso

Ana,9,A

Luis,6,B

Mia,10,A

Leo,7,B

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>pd.read_csv()</code>	Lee un archivo CSV.	<code>pd.read_csv("notas.csv")</code>	<pre> nombre nota curso 0 Ana 9 A 1 Luis 6 B 2 Mia 10 A 3 Leo 7 B </pre>
<code>sep=";"</code>	CSV con punto y coma (Excel en español).	<code>pd.read_csv("notas_pc.csv", sep=";")</code>	<pre> nombre nota 0 Ana 9 1 Luis 6 </pre>
<code>usecols=</code>	Solo ciertas columnas.	<code>pd.read_csv("notas.csv", usecols=["nombre", "nota"])</code>	<pre> nombre nota 0 Ana 9 1 Luis 6 2 Mia 10 3 Leo 7 </pre>
<code>nrows=</code>	Solo las primeras filas.	<code>pd.read_csv("notas.csv", nrows=2)</code>	<pre> nombre nota curso 0 Ana 9 A 1 Luis 6 B </pre>
<code>index_col=</code>	Una columna como índice.	<code>pd.read_csv("notas.csv", index_col="nombre").head(2)</code>	<pre> nota curso nombre Ana 9 A Luis 6 B </pre>
<code>header=None</code>	El archivo no tiene encabezados.	<code>pd.read_csv("notas.csv", header=None).head(2)</code>	<pre> 0 1 2 0 nombre nota curso 1 Ana 9 A </pre>
<code>encoding=</code>	Para tildes y ñ.	<code>pd.read_csv("notas.csv", encoding="utf-8").shape</code>	(4, 3)
<code>pd.read_excel()</code>	Lee un archivo Excel.	<code>pd.read_excel("notas.xlsx")</code>	<pre> nombre nota curso 0 Ana 9 A 1 Luis 6 B 2 Mia 10 A 3 Leo 7 B </pre>
<code>sheet_name=</code>	Elige la hoja del Excel.	<code>pd.read_excel("notas.xlsx", sheet_name=0).shape</code>	(4, 3)
<code>df.to_csv()</code>	Guarda en CSV.	<pre> df = pd.read_csv("notas.csv") df.to_csv("copia.csv", index=False) print(open("copia.csv").read()) </pre>	<pre> nombre,nota,curso Ana,9,A Luis,6,B Mia,10,A Leo,7,B </pre>
<code>df.to_excel()</code>	Guarda en Excel.	<pre> import os df = pd.read_csv("notas.csv") df.to_excel("salida.xlsx", index=False) print(os.path.exists("salida.xlsx")) </pre>	True

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Desde internet	Acepta un enlace.	<code>pd.read_csv("https://sitio.com/datos.csv")</code>	Descarga el CSV y lo carga como DataFrame
Archivo inexistente	Revisa la ruta del archivo.	<code>pd.read_csv("nada.csv")</code>	FileNotFoundError: [Errno 2] No such file or directory: 'nada.csv'

TEMA 66

Pandas: explorar y limpiar

Antes de analizar hay que limpiar: encontrar vacíos (NaN), llenarlos o borrarlos, quitar filas repetidas, renombrar columnas y ordenar. Casi todos los métodos devuelven un DataFrame nuevo (no modifican el original).

RECUERDA Limpiar = lavar los datos antes de cocinar: sin vacíos (NaN), sin repetidos y con nombres claros.

DATOS DE EJEMPLO

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    "nombre": ["Ana", "Luis", "Mia", "Mia"],
    "nota": [9, np.nan, 10, 10]})
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
df.isna()	True donde hay vacíos.	<code>df.isna()</code>	<pre> nombre nota 0 False False 1 False True 2 False False 3 False False</pre>
isna().sum()	Cuántos vacíos por columna.	<code>df.isna().sum()</code>	<pre> nombre 0 nota 1 dtype: int64</pre>
df.dropna()	Borra filas con vacíos.	<code>df.dropna()</code>	<pre> nombre nota 0 Ana 9.0 2 Mia 10.0 3 Mia 10.0</pre>
df.fillna(0)	Rellena vacíos con un valor.	<code>df.fillna(0)</code>	<pre> nombre nota 0 Ana 9.0 1 Luis 0.0 2 Mia 10.0 3 Mia 10.0</pre>
Rellenar con el promedio	Lo más usado en notas.	<code>df["nota"].fillna(df["nota"].mean())</code>	<pre> 0 9.000000 1 9.666667 2 10.000000 3 10.000000 Name: nota, dtype: float64</pre>
df.duplicated()	Marca filas repetidas.	<code>df.duplicated()</code>	<pre> 0 False 1 False 2 False 3 True dtype: bool</pre>
df.drop_duplicates()	Quita filas repetidas.	<code>df.drop_duplicates()</code>	<pre> nombre nota 0 Ana 9.0 1 Luis NaN 2 Mia 10.0</pre>
df.rename()	Cambia nombres de columnas.	<code>df.rename(columns={"nota": "puntaje"})</code>	<pre> nombre puntaje 0 Ana 9.0 1 Luis NaN 2 Mia 10.0 3 Mia 10.0</pre>

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>drop(columns=)</code>	Elimina una columna.	<code>df.drop(columns="nota")</code>	<pre> nombre 0 Ana 1 Luis 2 Mia 3 Mia </pre>
<code>astype()</code>	Cambia el tipo de una columna.	<code>df["nota"].fillna(0).astype(int)</code>	<pre> 0 9 1 0 2 10 3 10 Name: nota, dtype: int64 </pre>
<code>sort_values()</code>	Ordena por una columna.	<code>df.sort_values("nota", ascending=False)</code>	<pre> nombre nota 2 Mia 10.0 3 Mia 10.0 0 Ana 9.0 1 Luis NaN </pre>
<code>unique()</code>	Valores distintos.	<code>list(df["nombre"].unique())</code>	<code>['Ana', 'Luis', 'Mia']</code>
<code>value_counts()</code>	Cuántas veces aparece cada uno.	<code>df["nombre"].value_counts()</code>	<pre> nombre Mia 2 Ana 1 Luis 1 Name: count, dtype: int64 </pre>
<code>str.upper()</code>	Texto en mayúsculas.	<code>df["nombre"].str.upper().tolist()</code>	<code>['ANA', 'LUIS', 'MIA', 'MIA']</code>
<code>reset_index(drop=True)</code>	Reinicia el índice (0, 1, 2...).	<code>df.dropna().reset_index(drop=True)</code>	<pre> nombre nota 0 Ana 9.0 1 Mia 10.0 2 Mia 10.0 </pre>
<code>replace()</code>	Cambia un valor por otro.	<code>df.replace(10, 100)</code>	<pre> nombre nota 0 Ana 9.0 1 Luis NaN 2 Mia 100.0 3 Mia 100.0 </pre>

TEMA 67

Pandas: seleccionar y filtrar

Se elige una columna con `df["col"]`, varias con `df[["a", "b"]]`, filas por etiqueta con `loc` y por posición con `iloc`. Para filtrar se usa una condición dentro de corchetes: `df[df["nota"] > 7]`. Combina condiciones con `&` y `|` entre paréntesis.

RECUERDA `loc` = por letrero (etiqueta). `iloc` = por posición (número). Y para filtrar: `df[condición]`.

DATOS DE EJEMPLO

```

import pandas as pd
df = pd.DataFrame({
    "nombre": ["Ana", "Luis", "Mia", "Leo"],
    "nota": [9, 6, 10, 7],
    "curso": ["A", "B", "A", "B"]})

```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>df["col"]</code>	Una columna.	<code>df["nota"]</code>	<pre> 0 9 1 6 2 10 3 7 Name: nota, dtype: int64 </pre>
<code>df[["a", "b"]]</code>	Varias columnas.	<code>df[["nombre", "nota"]]</code>	<pre> nombre nota 0 Ana 9 1 Luis 6 2 Mia 10 3 Leo 7 </pre>

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>df.loc[0]</code>	Una fila por etiqueta.	<code>df.loc[0]</code>	<pre> nombre Ana nota 9 curso A Name: 0, dtype: object </pre>
<code>df.loc[0, "nombre"]</code>	Una celda por etiquetas.	<code>df.loc[0, "nombre"]</code>	'Ana'
<code>df.iloc[1]</code>	Una fila por posición.	<code>df.iloc[1]</code>	<pre> nombre Luis nota 6 curso B Name: 1, dtype: object </pre>
<code>df.iloc[0:2]</code>	Rango de filas por posición.	<code>df.iloc[0:2]</code>	<pre> nombre nota curso 0 Ana 9 A 1 Luis 6 B </pre>
<code>df.iloc[0, 1]</code>	Una celda por posición.	<code>df.iloc[0, 1]</code>	9
<code>df[df["nota"] > 7]</code>	Filtro con una condición.	<code>df[df["nota"] > 7]</code>	<pre> nombre nota curso 0 Ana 9 A 2 Mia 10 A </pre>
<code>& (y)</code>	Dos condiciones a la vez.	<code>df[(df["nota"] > 6) & (df["curso"] == "A")]</code>	<pre> nombre nota curso 0 Ana 9 A 2 Mia 10 A </pre>
<code> (o)</code>	Una u otra condición.	<code>df[(df["nota"] < 7) (df["nota"] == 10)]</code>	<pre> nombre nota curso 1 Luis 6 B 2 Mia 10 A </pre>
<code>df.query()</code>	Filtro escrito como texto.	<code>df.query("nota >= 7")</code>	<pre> nombre nota curso 0 Ana 9 A 2 Mia 10 A 3 Leo 7 B </pre>
<code>isin()</code>	Está en esta lista.	<code>df[df["curso"].isin(["A"])]</code>	<pre> nombre nota curso 0 Ana 9 A 2 Mia 10 A </pre>
<code>str.startswith()</code>	Texto que empieza con...	<code>df[df["nombre"].str.startswith("L")]</code>	<pre> nombre nota curso 1 Luis 6 B 3 Leo 7 B </pre>
<code>nlargest()</code>	Las n filas con mayor valor.	<code>df.nlargest(2, "nota")</code>	<pre> nombre nota curso 2 Mia 10 A 0 Ana 9 A </pre>
<code>loc[condición, col]</code>	Filtra y toma una columna.	<code>df.loc[df["nota"] > 7, "nombre"].tolist()</code>	['Ana', 'Mia']
Nueva columna	Se crea al asignarla.	<code>df["aprobado"] = df["nota"] >= 7</code> <code>print(df)</code>	<pre> nombre nota curso 0 Ana 9 A True 1 Luis 6 B False 2 Mia 10 A True 3 Leo 7 B True </pre>
Operar una columna	Se aplica a toda la columna.	<code>df["nota"] + 1</code>	<pre> 0 10 1 7 2 11 3 8 Name: nota, dtype: int64 </pre>
Columna inexistente	Da KeyError.	<code>df["edad"]</code>	KeyError: 'edad'

Pandas: agrupar y analizar

groupby() agrupa las filas por una columna y luego se calcula algo sobre cada grupo (media, suma, conteo...). También hay estadísticas directas, tablas dinámicas (pivot_table) y unión de tablas (merge, concat).

RECUERDA groupby = separar en montones por curso, calcular un número por montón y mostrar un resumen.

DATOS DE EJEMPLO

```
import pandas as pd
df = pd.DataFrame({
    "nombre": ["Ana", "Luis", "Mia", "Leo"],
    "nota": [9, 6, 10, 7],
    "curso": ["A", "B", "A", "B"]})
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
mean()	Promedio de la columna.	df["nota"].mean()	8.0
median()	Mediana.	df["nota"].median()	8.0
sum() / count()	Suma y cantidad.	print(df["nota"].sum(), df["nota"].count())	32 4
min() / max()	Menor y mayor.	print(df["nota"].min(), df["nota"].max())	6 10
std()	Desviación estándar.	round(df["nota"].std(), 2)	1.83
idxmax()	Fila con el mayor valor.	df.loc[df["nota"].idxmax(), "nombre"]	'Mia'
groupby().mean()	Promedio por grupo.	df.groupby("curso") ["nota"].mean()	curso A 9.5 B 6.5 Name: nota, dtype: float64
groupby().max()	Máximo por grupo.	df.groupby("curso")["nota"].max()	curso A 10 B 7 Name: nota, dtype: int64
groupby().size()	Cantidad de filas por grupo.	df.groupby("curso").size()	curso A 2 B 2 dtype: int64
groupby().agg()	Varias estadísticas a la vez.	df.groupby("curso") ["nota"].agg(["min", "max"])	min max curso A 9 10 B 6 7
value_counts()	Frecuencia de cada valor.	df["curso"].value_counts()	curso A 2 B 2 Name: count, dtype: int64
pivot_table()	Tabla dinámica.	df.pivot_table(values="nota", index="curso", aggfunc="mean")	nota curso A 9.5 B 6.5
df.agg()	Estadísticas elegidas.	df.agg({"nota": ["mean", "max"]})	nota mean 8.0 max 10.0
apply()	Aplica una función a cada valor.	df["nota"].apply(lambda n: n * 10)	0 90 1 60 2 100 3 70 Name: nota, dtype: int64

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>between()</code>	¿Está entre dos valores?	<code>df["nota"].between(7, 9)</code>	<pre>0 True 1 False 2 False 3 True Name: nota, dtype: bool</pre>
<code>rank()</code>	Posición en el ranking.	<code>df["nota"].rank(ascending=False)</code>	<pre>0 2.0 1 4.0 2 1.0 3 3.0 Name: nota, dtype: float64</pre>
<code>cumsum()</code>	Suma acumulada.	<code>df["nota"].cumsum()</code>	<pre>0 9 1 15 2 25 3 32 Name: nota, dtype: int64</pre>
<code>merge()</code>	Une tablas por una columna.	<pre>extra = pd.DataFrame({"curso": ["A", "B"], "profe": ["Paz", "Ruiz"]}) df.merge(extra, on="curso")</pre>	<pre> nombre nota curso profe 0 Ana 9 A Paz 1 Luis 6 B Ruiz 2 Mia 10 A Paz 3 Leo 7 B Ruiz</pre>
<code>concat()</code>	Apila tablas una sobre otra.	<code>pd.concat([df, df]).shape</code>	(8, 3)

TEMA 69

Matplotlib: gráficos básicos

Matplotlib dibuja gráficos. Se usa el módulo pyplot, importado como plt. Primero se crea el gráfico (plot, bar, scatter, hist...) y al final se muestra con plt.show().

RECUERDA Cada gráfico es una pregunta: ¿cómo cambia en el tiempo? → líneas. ¿Quién es mayor? → barras. ¿Hay relación? → dispersión. ¿Cómo se reparten? → histograma.

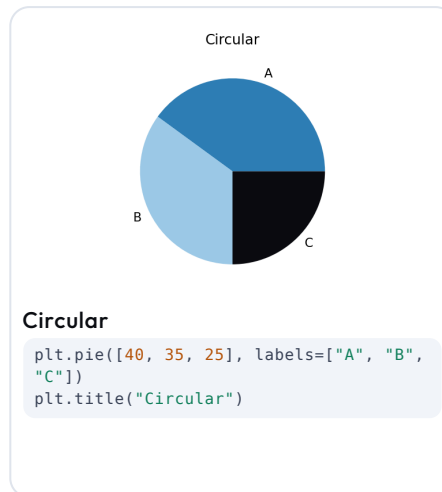
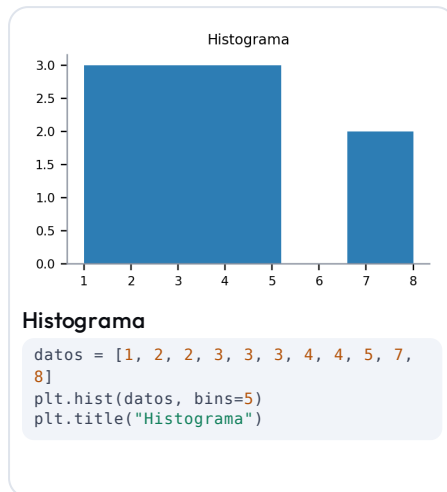
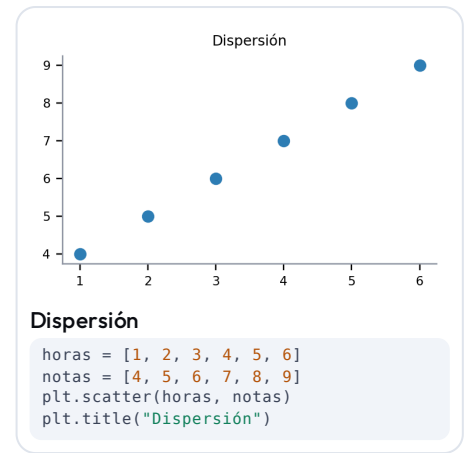
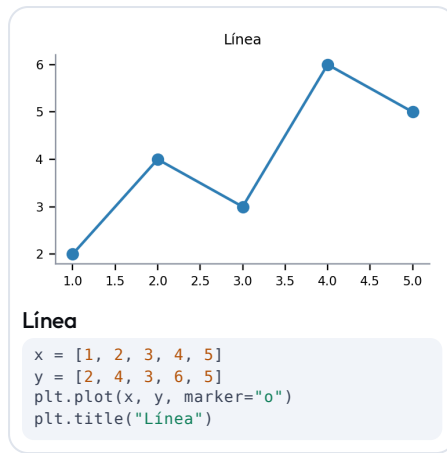
DATOS DE EJEMPLO

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 4, 3, 5]
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>plt.plot(x, y)</code>	Gráfico de líneas (evolución).	<code>plt.plot(x, y)</code>	Línea que une los puntos
<code>plt.bar()</code>	Barras verticales (comparar).	<code>plt.bar(["A", "B", "C"], [5, 3, 7])</code>	Una barra por categoría
<code>plt.barh()</code>	Barras horizontales.	<code>plt.barh(["A", "B", "C"], [5, 3, 7])</code>	Barras acostadas
<code>plt.scatter()</code>	Dispersión (relación entre dos datos).	<code>plt.scatter(x, y)</code>	Un punto por pareja (x, y)
<code>plt.hist()</code>	Histograma (cómo se reparten).	<code>plt.hist([1, 2, 2, 3, 3, 3, 4], bins=4)</code>	Barras de frecuencia
<code>plt.pie()</code>	Gráfico circular (partes de un total).	<code>plt.pie([40, 35, 25], labels=["A", "B", "C"])</code>	Círculo dividido en porciones
<code>plt.boxplot()</code>	Caja y bigotes (datos atípicos).	<code>plt.boxplot([1, 2, 2, 3, 4, 9])</code>	Caja con mediana y valores extremos
<code>plt.show()</code>	Muestra el gráfico en pantalla.	<code>plt.show()</code>	Abre la ventana con el gráfico

Los gráficos de ejemplo están al final de este tema.

Gráficos de ejemplo



TEMA 70

Matplotlib: personalizar y guardar

Un gráfico se vuelve útil cuando tiene título, nombres de ejes, leyenda y colores claros. Estas funciones se llaman antes de `plt.show()`. Para guardar la imagen se usa `plt.savefig()` ANTES de `plt.show()`.

RECUERDA Un gráfico sin título ni ejes es un dibujo sin explicación. Siempre: título, ejes y leyenda.

DATOS DE EJEMPLO

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 4, 3, 5]
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>plt.title()</code>	Título del gráfico.	<code>plt.title("Notas del curso")</code>	Título arriba del gráfico
<code>plt.xlabel()</code> / <code>plt.ylabel()</code>	Nombres de los ejes.	<code>plt.xlabel("Alumno")</code> <code>plt.ylabel("Nota")</code>	Texto en el eje x y en el eje y
<code>label= +</code> <code>plt.legend()</code>	Leyenda de varias series.	<code>plt.plot(x, y, label="Ana")</code> <code>plt.legend()</code>	Cuadro con el nombre de cada línea
<code>plt.grid(True)</code>	Cuadrícula de fondo.	<code>plt.grid(True)</code>	Líneas de guía en el fondo
<code>color, marker, linestyle</code>	Estilo de la línea.	<code>plt.plot(x, y, color="red", marker="o", linestyle="--")</code>	Línea roja punteada con círculos
<code>alpha=</code>	Transparencia (0 a 1).	<code>plt.scatter(x, y, alpha=0.5)</code>	Puntos semitransparentes
<code>plt.xlim()</code> / <code>plt.ylim()</code>	Límites de los ejes.	<code>plt.xlim(0, 10)</code> <code>plt.ylim(0, 8)</code>	Eje x de 0 a 10 y eje y de 0 a 8

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>plt.xticks(rotation=)</code>	Gira las etiquetas del eje x.	<code>plt.xticks(rotation=45)</code>	Etiquetas inclinadas 45°
<code>plt.axhline()</code>	Línea horizontal de referencia.	<code>plt.axhline(y=3, color="gray")</code>	Raya gris en y = 3
<code>plt.text()</code>	Escribe texto sobre el gráfico.	<code>plt.text(2, 4, "máximo")</code>	La palabra «máximo» en (2, 4)
<code>plt.figure(figsize=)</code>	Tamaño del gráfico en pulgadas.	<code>plt.figure(figsize=(6, 4))</code>	Lienzo de 6 x 4 pulgadas
<code>plt.subplots()</code>	Varios gráficos en una figura.	<code>fig, ax = plt.subplots(1, 2)</code>	Dos gráficos lado a lado
<code>plt.tight_layout()</code>	Acomoda los márgenes.	<code>plt.tight_layout()</code>	Evita que se encimen los textos
<code>plt.style.use()</code>	Estilos prediseñados.	<code>plt.style.use("ggplot")</code>	Cambia colores y fondo
<code>plt.savefig()</code>	Guarda el gráfico como imagen.	<code>plt.savefig("grafico.png")</code>	Crea el archivo grafico.png
<code>plt.close()</code>	Cierra la figura.	<code>plt.close()</code>	Libera la memoria